

Teoria Algoritmi

CAPITOLO 1: INTRODUZIONE

Questo capitolo fissa il vocabolario di base. Sembra banale, ma nasconde i concetti su cui si poggiano le valutazioni di complessità.

La Teoria (Zero Omissioni & Matematica Umana)

- **Definizione di Algoritmo:** È una sequenza di passi finiti e non ambigui che trasforma un *input* in un *output*. Un algoritmo si dice **corretto** se, per *qualsiasi* input, termina restituendo l'output desiderato. Se non termina, o restituisce un output sbagliato anche solo per un caso, non è corretto.
- **Efficienza:** Misurare il tempo in secondi non ha senso (dipende dal PC). Usiamo una funzione matematica: $Dimensione \rightarrow \mathbb{R}^{>0}$. Significa che prendiamo la dimensione dell'input (chiamata n) e le associamo un numero reale positivo che rappresenta il "costo" (numero di istruzioni base eseguite).
- **L'esempio della Moltiplicazione tra Matrici:** L'autore analizza lo pseudocodice di una moltiplicazione riga per riga per calcolare il costo esatto. Il risultato è un polinomio lunghissimo (es. $cnlm + 3nlm + \dots$).
 - *La Traduzione Umana:* Di tutto quel mostro matematico, a noi interessa solo il termine che cresce più velocemente (il **termine dominante**, in questo caso $cnlm$). Le costanti (come il 3 o il +4 finale) vengono ignorate perché, quando n diventa gigantesco (es. un milione), il peso di quelle costanti è come una goccia nell'oceano.
- **Tipi di Crescita:** Un algoritmo può avere una crescita *Lineare* (raddoppia l'input, raddoppia il tempo), *Esponenziale* (raddoppia l'input, il tempo esplode di una potenza), o *Logaritmica* (raddoppia l'input, il tempo aumenta solo di un singolo passo). Quella logaritmica è la più ambita.

Lente per le Crocette (Caccia al Trabocchetto)

- **La trappola delle costanti:** Nelle affermazioni V/F, potrebbero scriverti: "*Le costanti vengono sempre ignorate quando si valuta un algoritmo*". È **FALSO**. Le costanti vengono ignorate *nell'analisi asintotica* (per grandi n), ma se devi scegliere tra due algoritmi dello *stesso ordine* (es. due algoritmi $O(n)$), vince quello con la costante più piccola.

Lente per i Problemi (Il Kit di Costruzione)

- Quando nella Seconda Parte scriverai il tuo pseudocodice, non dovrai *mai* fare il conto "riga per riga" come nel Capitolo 1. Ti basterà contare i cicli annidati. Hai un `for` che va da 1 a V , e dentro un altro `for` che va da 1 a V ? Il costo è $O(V^2)$. Fine.

CAPITOLO 2: NOTAZIONE ASINTOTICA

Qui c'è il 90% della matematica del primo blocco. Serve a classificare la velocità degli algoritmi in "scatole" ben precise.

La Teoria (Zero Omissioni & Matematica Umana)

L'analisi asintotica studia come si comporta l'algoritmo da un certo punto in poi (chiamato $\bar{n}$), andando verso l'infinito.

Ci sono tre notazioni fondamentali. Immagina di voler descrivere l'altezza di una persona ($f(n)$) rispetto a un muro di riferimento ($g(n)$). Il valore c è solo un "fattore di scala" (un moltiplicatore) per alzare o abbassare il muro.

1. Notazione O (Big-O) - Il Soffitto (Limite Superiore)

- **Significato:** La funzione $f(n)$ cresce al massimo come $g(n)$. Non andrà mai peggio di così.
- **Matematica:** $\exists c, \bar{n} \text{ t.c. } \forall n > \bar{n}, f(n) \leq c \cdot g(n)$.
 - *Umano:* Esiste un momento $\bar{n}$ a partire dal quale la nostra funzione starà *sempre* sotto o sarà uguale al muro $g(n)$ moltiplicato per una costante.
- **Esempio:** $n \in O(n^2)$ è VERO, perché n sta comodamente sotto il soffitto di n^2 .

2. Notazione Ω (Big-Omega) - Il Pavimento (Limite Inferiore)

- **Significato:** La funzione $f(n)$ cresce *almeno* come $g(n)$. Non andrà mai meglio di così.
- **Matematica:** $\exists c > 0, \bar{n} \text{ t.c. } \forall n > \bar{n}, f(n) \geq c \cdot g(n)$.
 - *Umano:* Da un certo punto in poi, la nostra funzione starà *sempre* sopra il pavimento.

3. Notazione Θ (Theta) - Il Sandwich (Limite Stretto)

- **Significato:** La funzione $f(n)$ cresce *esattamente* allo stesso ritmo di $g(n)$.
- **Matematica:** $\exists c_1, c_2, \bar{n} \text{ t.c. } \forall n \geq \bar{n}, c_1 g(n) \leq f(n) \leq c_2 g(n)$.
 - *Umano:* Posso scalare il muro $g(n)$ per creare un pavimento (c_1) e un soffitto (c_2). La mia funzione ci passerà in mezzo per sempre.

4. Esercizi e Dimostrazioni Matematiche

Gli appunti presentano due dimostrazioni algebriche, traduciamo il concetto logico:

- **Regola dell'inverso:** $f \in O(g) \iff g \in \Omega(f)$. *Umano:* Se io sono al massimo alto quanto te (O), tu sei almeno alto quanto me (Ω).
- **Regola della somma:** Se hai due funzioni che sono entrambe $O(g)$, la loro somma sarà ancora $O(g)$. *Umano:* Se fai un ciclo da $O(n)$ e poi ne fai un altro staccato da

$O(n)$, il tempo totale è $2n$. Togliendo la costante 2, la scatola finale rimane $O(n)$. Non diventa n^2 !

- **Definizione con i Limiti:** Invece di usare c e $\bar{n}$, puoi usare l'Analisi 1 facendo $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$. Se il limite è finito, è O . Se è > 0 , è Ω . Se è un numero tra 0 e infinito, è Θ .

5. Relazione tra Algoritmo e Problema (CRITICO)

Questa è la distinzione più sottile e importante del capitolo:

- **Per un ALGORITMO (A):**
 - $A \in O(f)$: L'algoritmo nel *caso peggiore in assoluto* non supera il tempo f .
 - $A \in \Omega(f)$: Esiste *almeno un input sfortunato* che costringe l'algoritmo a impiegare il tempo f . (Attenzione: non significa che ci mette *sempre* f).
- **Per un PROBLEMA (P):** (Es. il problema in astratto di ordinare dei numeri).
 - $P \in O(f)$: Esiste *almeno un* algoritmo in grado di risolvere il problema nel tempo f (es. il problema dell'ordinamento è $O(n \log n)$ perché qualcuno ha inventato il Merge Sort).
 - $P \in \Omega(f)$: **TUTTI** gli algoritmi passati, presenti e futuri che cercheranno di risolvere questo problema ci metteranno *almeno* il tempo f . È un muro fisico e matematico insuperabile.

Lente per le Crocette (Caccia al Trabocchetto)

- **Il Tranello Algoritmo vs Problema:** Troverai al 100% una crocetta così: "*Il problema dell'ordinamento basato su confronti appartiene a $O(n^2)$* ". È **VERA**. Essendo il limite inferiore assoluto $\Omega(n \log n)$, il problema può benissimo stare "sotto il soffitto" di n^2 . Ma se ti scrivono "*Il problema dell'ordinamento appartiene a $\Omega(n^2)$* " è **FALSA**, perché noi sappiamo che si può fare di meglio (in $n \log n$).
- **Il tranello dei Limiti non Stretti:** $n = O(n^{100})$ è **VERA** dal punto di vista matematico. O significa "minore o uguale". Non deve essere per forza un limite stretto (quello è Θ).
- **Il tranello polinomiale:** $4n^3 \in O(2n^2)$ è **FALSA**. Una funzione cubica sfonda sempre e comunque il soffitto di una quadratica, per quanto grande tu possa scegliere la costante c per moltiplicare n^2 . Non si batte la matematica degli esponenti.

Lente per i Problemi (Il Kit di Costruzione)

- Quando alla fine del maxi-problema ti verrà chiesto di "studiare la complessità", tu userai **SEMPRE la notazione O (Big-O)**. Stai fornendo una garanzia: "Professore, il mio algoritmo che ho appena inventato, anche se la sfortuna ci vede benissimo, al massimo impiegherà $O(V \log V + E)$ ". Non usare mai Ω o Θ per descrivere la

complessità di una tua soluzione mista in sede d'esame, a meno che tu non stia descrivendo un banalissimo ciclo `for` che scansiona sempre tutto a prescindere.

Prevenzione delle Sorprese (Oltre i PDF)

- *Condizione Necessaria e Sufficiente:* Perché $f(n)$ sia $\Theta(g(n))$, è *necessario e sufficiente* che sia contemporaneamente $O(g(n))$ e $\Omega(g(n))$. Se ti chiedono se Θ esiste sempre, la risposta è NO. Se hai un algoritmo che nel caso migliore fa 1 operazione e nel caso peggiore n operazioni, il suo comportamento oscilla. In questo caso l'algoritmo è $O(n)$ e $\Omega(1)$, ma **non esiste un Θ globale** per l'algoritmo perché soffitto e pavimento non coincidono.

CAPITOLO 3: DIVIDE ET IMPERA E RICORRENZE

Il paradigma "Divide et Impera" (Divide and Conquer) è una strategia di problem-solving basata su tre fasi distinte:

1. **Divide:** Si spezza il problema originale in tanti sotto-problemi più piccoli, ma della stessa identica natura.
2. **Impera (Conquista):** Si risolvono i sotto-problemi in modo ricorsivo. Quando il problema è diventato così piccolo da essere banale, si risolve direttamente (Caso Base).
3. **Combina:** Si prendono le soluzioni dei sotto-problemi e le si "incollano" insieme per ottenere la soluzione del problema di partenza.

Un algoritmo che usa questa tecnica genera una **Equazione di Ricorrenza**, ovvero un'equazione matematica che descrive il tempo di esecuzione in funzione della sua stessa esecuzione su input più piccoli.

Esempio Umano: Per calcolare il fattoriale di n , ci metto il tempo per fare una moltiplicazione più il tempo che ci metto a calcolare il fattoriale di $n - 1$. In formule:
$$T(n) = 1 + T(n - 1).$$

Per risolvere (cioè trovare il Big-O o il Theta di) queste equazioni, abbiamo **tre metodi**.

1. Metodo di Sostituzione (L'intuizione matematica)

Consiste in due passi: indovinare la forma della soluzione e usare l'induzione matematica per dimostrare che l'ipotesi è corretta, trovando le costanti giuste.

- **La Teoria:** Se ho $T(n) = 2T(\lfloor n/2 \rfloor) + n$ e qualcuno mi "sussurra" che la soluzione è $O(n \log n)$, io sostituisco questa ipotesi nell'equazione e verifico se esiste una costante c che fa tornare i conti.
- **Il Trucco del termine di ordine inferiore:** A volte la matematica fallisce per colpa di un banale $+1$. Nel caso di $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$, ipotizzare $T(n) \leq cn$ porta a un vicolo cieco ($cn + 1 \leq cn$ è impossibile). Il trucco sta nell'ipotizzare

una formula più forte: $T(n) \leq cn - b$ (sottraendo una costante). Così facendo l'induzione funziona perfettamente.

2. Metodo dell'Albero di Ricorsione (La visualizzazione)

Si converte l'equazione in un albero per "vedere" dove si concentra lo sforzo dell'algoritmo.

- **La Teoria:** Ogni nodo rappresenta il costo di un sotto-problema. Si somma il costo orizzontalmente (livello per livello) e poi si somma il totale di tutti i livelli in verticale.
- *Matematica Umana:* Se hai $T(n) = T(n/3) + T(2n/3) + n$, disegni un nodo padre che costa n . Da lui partono due figli, uno costa $n/3$ e uno $2n/3$. La somma del primo livello è n . La somma del secondo livello sarà ancora n . Se so quanti livelli ha l'albero (la sua altezza, che è determinata dal ramo più lungo, in questo caso $\log_{3/2} n$), posso calcolare il costo totale.

3. Il Metodo Principale o Master Theorem (L'arma suprema)

Questo è il cuore pulsante del capitolo. Fornisce una "ricetta" istantanea per risolvere ricorrenze che si presentano nella forma rigorosa:

$$T(n) = aT(n/b) + f(n)$$

Dove $a \geq 1$ e $b > 1$.

Matematica tradotta in "Umano" (Come leggere la formula):

- a : In quanti sotto-problemi spacco l'array? (Quanti cloni di me stesso evoco).
- b : Di quanto restringo il problema ogni volta? (Es. $b = 2$ significa che divido a metà).
- $f(n)$: Quanto mi costa la fase di "Divide" e la fase di "Combina"? (Quanto mi costa rompere l'array e incollare i risultati).
- $n^{\log_b a}$: È il "peso delle foglie". Rappresenta la mole di lavoro che si accumula alla base dell'albero di ricorsione (i casi base).

Il teorema è una **sfida di forza** tra il lavoro per ricombinare i pezzi ($f(n)$) e il lavoro per calcolare i casi base ($n^{\log_b a}$).

- **Caso 1 (Vincono le foglie):** Se il peso delle foglie $n^{\log_b a}$ è polinomialmente PIÙ GRANDE di $f(n)$, allora il tempo totale è dominato dalle foglie: $T(n) = \Theta(n^{\log_b a})$.
 - *L'Epsilon (ϵ):* La differenza non deve essere sottile. Le foglie devono vincere con un distacco di almeno n^ϵ con $\epsilon > 0$.
- **Caso 2 (Pareggio perfetto):** Se $f(n)$ e $n^{\log_b a}$ sono grandi uguali (Θ), pareggiano. Il costo totale è il valore del pareggio moltiplicato per il numero di livelli dell'albero ($\log n$). Risultato: $T(n) = \Theta(n^{\log_b a} \log n)$.

- **Caso 3 (Vince la ricombinazione):** Se $f(n)$ è polinomialmente PIÙ GRANDE delle foglie, e rispetta una condizione di regolarità (il costo dei figli è minore del costo del padre), allora il tempo totale è dominato da $f(n)$: $T(n) = \Theta(f(n))$.

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto dell'Applicabilità:** Ti scriveranno: "*Il Teorema Master si può applicare alla ricorrenza $T(n) = T(n-1) + n$* ". **FALSO.** Il teorema esige tassativamente che il problema venga *diviso* (n/b), non sottratto ($n-1$).
- **Il Trabocchetto del Buco Nero (ATTENZIONE QUI):** L'esame ama questa equazione: $T(n) = 2T(n/2) + n \log n$. Ti chiederanno se si può risolvere col Teorema Master. **FALSO.**
 - *Perché?* Calcoliamo le forze: $a = 2, b = 2$. Il peso delle foglie è $n^{\log_2 2} = n^1 = n$. La funzione di ricombinazione è $f(n) = n \log n$. Chi vince? Apparentemente $n \log n$ è più grande di n . Potresti pensare di essere nel Caso 3. MA il Teorema Master esige che la vittoria sia *polinomiale* (ci deve essere una differenza di almeno n^ϵ). Tra n e $n \log n$, il distacco è solo logaritmico, non polinomiale. Cadiamo in una zona morta in cui il teorema, nella sua forma base, fallisce miseramente!
- **Il Trabocchetto dei Decimali:** Ti scriveranno $T(n) = T(2n/3) + 1$. Qui la trappola è ricavare a e b . Ricorda che dividere per $3/2$ equivale a moltiplicare per $2/3$. Quindi $a = 1, b = 3/2$. Applicando il Caso 2, la soluzione è $\Theta(\log n)$.

Lente per i Problemi (Il Kit di Costruzione)

Nel blocco 2 dell'esame (il maxi-problema da 30 punti), il Teorema Master ti servirà come **preventivo istantaneo** del tuo algoritmo.

- **Lo Scenario:** Ti inventi un algoritmo Divide et Impera per un grafo o un array. Ad esempio, spezzi a metà i dati ($2T(n/2)$) e per riunire la soluzione usi un ciclo lineare ($+n$).
- **Il Calcolo Veloce:** Invece di fare dimostrazioni chilometriche sul foglio, scriverai: "*La complessità del mio algoritmo è descritta dalla ricorrenza $T(n) = 2T(n/2) + n$. Per il Teorema Master, poiché ci troviamo nel Caso 2 dove $n^{\log_2 2} = n$, il costo finale garantito è $\Theta(n \log n)$* ".

Il professore sa che padroneggi lo strumento e ti prenderai l'intero punteggio sulla complessità.

Prevenzione delle Sorprese (Oltre i PDF)

- **Cosa succede se $a < 1$?** Nelle crocette potrebbero piazzare $T(n) = 0.5T(n/2) + n$. Se rispondi applicando il teorema, hai perso. Il Teorema richiede categoricamente

che $a \geq 1$ (non puoi creare "mezzo" sotto-problema, algoritmicamente non ha senso fisico).

- **Il Caso 3 e la "Condizione di Regolarità":** Spesso sottovalutata, ma vitale. Per applicare il Caso 3 (quello in cui vince $f(n)$), non basta che $f(n)$ sia più grande. Devi verificare che $a \cdot f(n/b) \leq c \cdot f(n)$ per una costante $c < 1$. Significa che il tempo speso nei sotto-problemi combinati non deve esplodere, deve rimanere una frazione del tempo speso nel problema padre. In una crocetta subdola, un $f(n)$ che oscilla selvaggiamente (come $n^2 \sin n$) invaliderebbe il Caso 3!

IL MURO INVALIDICABILE: L'ALBERO DI DECISIONE E $\Omega(n \log n)$

Prima di parlare dei singoli algoritmi, gli appunti affrontano un limite fisico dell'informatica: **il problema dell'ordinamento basato su confronti**.

- **La Teoria:** Immagina di dover ordinare una sequenza $a_1, \dots, a_n$. Qualsiasi algoritmo che funziona "confrontando" due elementi alla volta per decidere chi è più grande può essere disegnato come un *Albero di Decisione*.
- **Matematica Umana:** Un array di n elementi può essere mescolato in $n!$ (n fattoriale) modi diversi (chiamate permutazioni). L'albero di decisione deve avere almeno una foglia per ogni possibile soluzione, quindi $n!$ foglie. L'altezza di questo albero rappresenta il numero di confronti da fare nel caso peggiore. Per la matematica degli alberi binari, l'altezza minima per avere $n!$ foglie è $\log(n!)$, che asintoticamente equivale a $\Omega(n \log n)$.

Lente per le Crocette (Trabocchetto Fisso):

Se leggi: "*Nessun algoritmo di ordinamento può fare meglio di $O(n \log n)$* ", è **FALSO**. La frase corretta è: "*Nessun algoritmo basato sui **confronti** può fare meglio di $\Omega(n \log n)$* ". Gli algoritmi lineari (che vedremo dopo) violano questa regola perché *non* usano i confronti!

GLI ALGORITMI BASATI SU CONFRONTI

Per ogni algoritmo devi tatuarti in mente tre parametri vitali: Complessità, Stabilità (mantiene l'ordine originale dei duplicati?) e In Loco (usa o no array di supporto enormi?).

1. Insertion Sort (L'ordinamento delle carte)

- **La Teoria:** Prende un elemento alla volta e lo "inserisce" scorrendo all'indietro nella porzione di array che ha già ordinato alla sua sinistra.
- **Matematica Umana e Costi:**
 - *Caso Peggior*e: L'array è ordinato al contrario. Per ogni elemento, deve scorrere tutto l'array all'indietro. Sommare $1 + 2 + 3 + \dots + n$ porta a $\Theta(n^2)$.

- *Caso Medio*: Metà dei confronti. Asintoticamente rimane $\Theta(n^2)$.
- *Caso Migliore*: L'array è già ordinato! Entra nel ciclo, vede che è tutto a posto ed esce subito: $O(n)$.
- 🔍 **Lente per le Crocette: In loco: Sì. Stabile: Sì.** È l'unico algoritmo che se gli dai un array già ordinato, ci mette pochissimo ($O(n)$).

2. Merge Sort (Il re del Divide et Impera)

- **La Teoria**: Taglia l'array esattamente a metà finché non restano singoli elementi, poi li "fonde" (Merge) a due a due rimettendoli in ordine.
- **Matematica Umana e Costi**: L'equazione è $T(n) = 2T(n/2) + \Theta(n)$. Per il Teorema Master, ricadiamo nel Caso 2, quindi costa **sempre** $\Theta(n \log n)$. Non ha un caso migliore o peggiore, taglia sempre a metà in modo cieco e metodico.
- 🔍 **Lente per le Crocette: Stabile: Sì. In loco: NO** (Il trabocchetto classico!). La funzione **Merge** ha bisogno di un array di supporto **B** grande quanto l'array originale per poter fondere i dati in sicurezza.

3. Quick Sort (L'illusionista)

- **La Teoria**: Sceglie un "Perno" (Pivot) e fa una **Partition**: butta tutti i minori del pivot a sinistra, tutti i maggiori a destra. Poi si richiama sulle due metà.
- **Matematica Umana e Costi**:
 - *Caso Peggiore*: L'array è già ordinato e scegli come pivot il primo elemento. Finisci per spezzare l'array in una parte grande $n - 1$ e una grande 0. Risultato catastrofico: $\Theta(n^2)$.
 - *Caso Ottimo/Medio*: Il pivot capita nel mezzo. Tagli in due metà da $n/2$. Risultato: $\Theta(n \log n)$.
 - *Versione Randomized*: Sceglie il pivot a caso per schivare la sfortuna, abbassando la *probabilità* del caso pessimo, ma matematicamente il caso peggiore resta $\Theta(n^2)$.
- 🔍 **Lente per le Crocette: In loco: Sì. Stabile: NO** (la **Partition** lancia i numeri da destra a sinistra senza rispettare l'ordine originale).
- 🚨 **Prevenzione Sorprese (Tail Call Elimination)**: Essendo ricorsivo, Quick Sort riempie la memoria "Stack" del PC. Nel caso pessimo, lo stack esplode a $O(n)$. Come si risolve? Facendo la ricorsione *solo* sulla metà più piccola e iterando su quella più grande (Tail Call). Questo blinda lo stack a $O(\log n)$ nel caso pessimo. *Crocetta garantita.*

4. Heap Sort (La Coda di Priorità mascherata)

- **La Teoria:** Tratta un normale array come se fosse un albero binario quasi completo. Un "Max-Heap" è un albero in cui il padre è sempre $\geq$ dei figli. L'indice del figlio sinistro è $2i$, quello destro $2i + 1$. L'algoritmo crea l'Heap, prende la radice (il massimo), la butta in fondo all'array, riduce la dimensione e ripara l'albero (**Max-Heapify**).
- **Matematica Umana e Costi:**
 - **Max-Heapify** : Ripara un nodo facendolo scivolare verso il basso. Costo: $O(\log n)$ (altezza dell'albero).
 - **Build-Max-Heap** : Costruisce l'Heap da un array caotico. Costo: $O(n)$ (Crocetta insidiosissima! Non è $n \log n$, perché applicando **Max-Heapify** dal basso, la maggior parte dei nodi sono foglie e fanno zero strada. La somma geometrica converge a $O(n)$).
 - **Heap Sort** totale: $\Theta(n \log n)$ (estrae n elementi, ogni estrazione costa $\log n$).
- 🔍 **Lente per le Crocette: In loco: Sì. Stabile: NO.**

GLI ALGORITMI LINEARI (Oltre il Muro)

Questi tre algoritmi ignorano i confronti e battono la barriera di $\Omega(n \log n)$ a patto che l'input rispetti dei vincoli rigidi.

5. Counting Sort (Il pallottoliere)

- **La Teoria:** Conta quante volte compare un numero e calcola quanti numeri ci sono più piccoli di lui (accumulo). Usa queste informazioni per piazzare il numero direttamente nella sua posizione finale.
- **Vincolo vitale:** Funziona solo se i numeri sono interi compresi in un range stretto da 1 a k .
- **Matematica Umana e Costi:** Crea un array di supporto grande k . Scorre l'array di input grande n . Costo finale: $\Theta(n + k)$. Se $k = O(n)$ (cioè il range dei numeri è vicino alla quantità di numeri), allora diventa magicamente $\Theta(n)$.
- 🔍 **Lente per le Crocette: In loco: NO** (serve l'array per contare). **Stabile:** Sì (lo usiamo come mattoncino per il prossimo algoritmo). Se in una crocetta ti dicono che "con un range enorme e sparso ha complessità lineare", è **FALSO**. Se k esplode, la memoria e il tempo esplodono con lui.

6. Radix Sort (La macchina a schede perforate)

- **La Teoria:** Ordina i numeri cifra per cifra, partendo dalle Unità (Least Significant Digit - LSD) per finire con la cifra più significativa.

- **Il Segreto:** Per ordinare le singole colonne di cifre, Radix Sort DEVE chiamare un "aiutante" **STABILE** (di solito proprio il Counting Sort). Se non fosse stabile, ordinando le decine scombinerebbe il lavoro perfetto fatto prima sulle unità.
- **Matematica Umana:** Tempo $\Theta(d(n + k))$, dove d è il numero di cifre. Se d è una costante, diventa tempo lineare.
-  **Lente per le Crocette: In loco:** NO (perché usa il Counting Sort come aiutante). **Stabile:** Sì.

7. Bucket Sort (I Secchielli)

- **La Teoria:** Presuppone che l'input sia distribuito in modo **uniforme** nell'intervallo $[0, 1)$. Taglia l'intervallo in n secchielli uguali e ci butta dentro i numeri. Poi ordina i singoli secchielli (solitamente con Insertion Sort).
- **Matematica Umana e Costi:** Visto che la probabilità è uniforme, ogni secchiello avrà in media 1 elemento. L'Insertion Sort su 1 elemento costa $O(1)$. Tempo medio atteso: $\Theta(n)$.
-  **Lente per le Crocette:** Caso peggiore: $O(n^2)$. Perché? Se la sfortuna vuole che tutti i numeri finiscano nell'ultimo secchio, farai un Insertion Sort su tutto l'array, beccandoti un $\Theta(n^2)$!.

LENTE PER I PROBLEMI (Il Kit di Costruzione)

Nel Maxi-problema della Seconda Parte non ti chiederanno mai "Come funziona il Radix Sort?". Ma quando avrai un problema e dovrai ordinare qualcosa come primo passaggio (es. l'algoritmo di Kruskal sui grafi che ordina gli archi per peso), dirai semplicemente:

- *"Ordino l'array usando Merge Sort in tempo garantito $O(n \log n)$ ".* (Garanzia matematica inattaccabile).
- Se il problema dice: *"Avete in input un milione di voti studenteschi (che vanno da 18 a 30)".* Il tuo intuito si deve accendere: il range è minuscolo (solo 13 valori possibili, $k = 13$), mentre n è enorme. Allora scriverai: *"Poiché il range dei valori $k \ll n$, non uso un ordinamento per confronti, ma ordino i voti con Counting Sort in tempo lineare $O(n)$ ".* Boom. Dimostri di dominare la materia.

CAPITOLO 5: SELEZIONE

La Teoria (Zero Omissioni & Matematica Umana)

- **Il Problema della Selezione:** L'obiettivo è trovare la i -esima statistica d'ordine in un array di n elementi non ordinati. Se $i = 1$ cerchi il minimo, se $i = n$ cerchi il massimo, se cerchi il valore centrale stai cercando la **mediana**.
- *L'Approccio Naive:* Ordini tutto l'array (ad esempio con Merge Sort in $O(n \log n)$) e prendi l'elemento in posizione i . Ma noi non vogliamo sprecare tempo a ordinare,

vogliamo farlo in tempo asintotico lineare $\Theta(n)$.

1. Minimo e Massimo Simultanei

- Trovare *solo* il minimo (o solo il massimo) richiede scorrere l'array facendo $n - 1$ confronti.
- Per trovare *sia* minimo *sia* massimo, la logica di base direbbe di fare $(n - 1) + (n - 1) = 2n - 2$ confronti.
- *L'Ottimizzazione Umana*: Invece di confrontare ogni elemento col minimo e col massimo, **li confronti a coppie tra loro** (1 confronto). Il vincitore della coppia "sfida" il massimo attuale, il perdente "sfida" il minimo attuale (2 confronti).
- *Costo Finale*: Hai speso 3 confronti per elaborare 2 elementi. Il costo totale scende da $2n$ a $3\lfloor n/2 \rfloor$ confronti, risparmiando tempo.

2. RANDOMIZED-SELECT (Tempo atteso lineare)

- Usa la logica del "Divide et Impera" ereditata dal Quick Sort. Scegli un pivot a caso e partizioni l'array: i minori a sinistra, i maggiori a destra.
- *L'Intuizione*: A differenza del Quick Sort, non ti richiami su entrambe le metà! Se cerchi il 3° elemento più piccolo, e il tuo pivot dopo la partizione finisce in 8° posizione, sai con certezza matematica che il tuo obiettivo è nella metà sinistra. Scarti del tutto la metà destra e ti richiami solo a sinistra.
- *Costi*: Nel caso medio (o atteso), scartare metà array a ogni passo genera un albero di ricorsione il cui costo crolla a $\Theta(n)$.
- *Il Problema*: Se hai una sfortuna cronica (il pivot estratto a caso è sempre il massimo o il minimo), scarcerai solo 1 elemento a iterazione. Il tempo esplode al caso pessimo $\Theta(n^2)$.

3. SELECT (Tempo lineare nel caso peggiore)

- Questo algoritmo serve per "corazzare" il Randomized-Select ed eliminare il caso pessimo quadratico. Il suo scopo è trovare un pivot *perfetto* che garantisca di spezzare bene l'array.
- *I 5 Passaggi*:
 1. Dividi l'array in gruppetti di 5 elementi.
 2. Trova la mediana di ogni gruppetto (usando un banale Insertion Sort, costa solo $O(1)$ a gruppo).
 3. Usa l'algoritmo **SELECT** ricorsivamente sull'array delle mediane per trovare la "mediana delle mediane" (x).
 4. Usa x come pivot per partizionare l'array.
 5. Vai a sinistra o a destra in base a dove si trova l'elemento i che cerchi.

- *Matematica Umana (L'Equazione Maledetta)*: Perché tutto questo sforzo? Il pivot x è garanzia di equilibrio. La matematica dimostra che almeno il 30% degli elementi sarà più grande di x (escludendo i resti), e almeno il 30% sarà più piccolo ($\frac{3n}{10} - 6$).

L'equazione diventa:

$$T(n) = T(\lceil n/5 \rceil) + T(7n/10 + 6) + \Theta(n)$$

- $T(\lceil n/5 \rceil)$: costo per trovare la mediana delle mediane al Passo 3.
 - $T(7n/10 + 6)$: costo per cercare nella metà peggiore al Passo 5 (che al massimo è il 70% dell'array).
 - $\Theta(n)$: lavoro di contorno (partizionamento).
- *La Magia*: $\frac{1}{5} + \frac{7}{10} = \frac{9}{10}$. Poiché $\frac{9}{10}$ è strettamente minore di 1, la quantità di lavoro si restringe a ogni iterazione. La soluzione con il metodo per sostituzione dimostra che tutto collassa a $O(n)$ garantito.

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto Assoluto del Mediano**: Se ti scrivono "*Il problema della selezione del mediano di un array non ordinato appartiene a $O(\log n)$* ", è **FALSA** (ed è apparsa spesso nei tuoi esami!). Il limite teorico inferiore in array non ordinati è sempre $\Omega(n)$, perché devi fisicamente "guardare" ogni elemento almeno una volta.
- **"Garantito" vs "Atteso"**: Se una crocetta dice "*Randomized-Select trova il mediano in tempo $O(n)$ nel caso peggiore*", è **FALSA**. Il suo caso peggiore è $\Theta(n^2)$. Se lo dicono per l'algoritmo *SELECT*, allora è **VERA**.
- **La Statistica d'Ordine Dinamica**: A volte tentano di fregarti chiedendo: "*È possibile sapere in tempo $\Theta(n)$ se in un array esistono due elementi la cui somma è X ?*". Usando un ordinamento o una selezione furbi, si risponde spesso di sì, ma ricorda che la selezione pura (isolare un elemento) è sempre limitata a $\Theta(n)$ di base.
- **Costo Min-Max**: Se ti chiedono quanti confronti servono per trovare sia il minimo che il massimo, non cascare sul $2n$. La risposta corretta e ottimizzata è $3\lfloor n/2 \rfloor$.

Lente per i Problemi (Il Kit di Costruzione)

Nel maxi-problema della Seconda Parte dell'esame, l'algoritmo **SELECT** è il tuo "coltellino svizzero" segreto.

- **Lo Scenario**: Immagina che ti chiedano di separare "le k stanze del labirinto che costano di meno" dalle altre, o "le prime k macchine più efficienti" senza sprecare tempo.
- **Come incastrarlo**: Se scrivi "*Ordino tutto l'array in $O(n \log n)$ e prendo le prime k* ", perdi punti di efficienza. Se invece scrivi: "*Uso l'algoritmo *SELECT* in tempo garantito $\Theta(n)$ per trovare la k -esima statistica d'ordine. Poi uso questo elemento*

come pivot per partizionare in $\Theta(n)$ i k elementi minori da una parte e i restanti dall'altra".

Hai appena abbattuto il tempo asintotico di una fase del tuo algoritmo da $n \log n$ a n puro. I professori impazziscono per queste ottimizzazioni.

Prevenzione delle Sorprese (Oltre i PDF)

- **Perché gruppi da 5? (Il Trabocchetto Teorico):** C'è una domanda di sbarramento orale o da quiz infame: "Perché nell'algoritmo SELECT dividiamo in gruppi da 5 e non da 3?".
 - *La risposta spietata:* Se fai gruppi da 3, la percentuale di scarto nel caso peggiore cambia. L'equazione diventerebbe $T(n) = T(n/3) + T(2n/3) + \Theta(n)$. Se sommi le frazioni $\frac{1}{3} + \frac{2}{3}$ ottieni $1n$. Il lavoro non "restringe" a ogni passo, e l'algoritmo esploderebbe a $O(n \log n)$, perdendo la linearità. I gruppi da 5 sono il minimo matematico per mantenere la somma delle frazioni ($\frac{9}{10}$) minore di 1!

1. STRUTTURE DATI ELEMENTARI

Questi sono i contenitori lineari di base. Le operazioni devono essere, idealmente, istantanee.

La Teoria (Zero Omissioni & Matematica Umana)

- **Lo Stack (La Pila):** Segue la politica **LIFO** (Last In, First Out). L'ultimo elemento inserito è il primo ad uscire, esattamente come una pila di piatti.
 - *Operazioni:* **PUSH** (inserisce in cima), **POP** (rimuove dalla cima). Il puntatore **top** tiene traccia di dove siamo.
 - *Costi:* Entrambe le operazioni costano $O(1)$.
 - *Errori:* Se fai **POP** su una pila vuota, hai un errore di *Underflow*; se fai **PUSH** su una pila piena, hai *Overflow*.
- **La Coda (Queue):** Segue la politica **FIFO** (First In, First Out). Il primo che entra è il primo a uscire, come alle poste.
 - *Operazioni:* **ENQUEUE** (inserisce alla fine, puntatore **tail**), **DEQUEUE** (rimuove dall'inizio, puntatore **head**).
 - *Costi:* Entrambe costano $O(1)$.
- **Le Liste Concatenate:** Gli elementi non sono vicini in memoria come negli array. Ogni "nodo" contiene il dato (**key**) e un puntatore al prossimo nodo (**next**). Se ha anche un puntatore all'indietro (**prev**), si dice **doppiamente concatenata**.

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto dell'Array Circolare:** Se una crocetta ti chiede: "*L'operazione di DEQUEUE in una coda implementata con array costa sempre $O(1)$* ". È VERO solo se l'array è gestito in modo **circolare** (come insegna la teoria standard). Se invece per togliere il primo elemento shifti tutti gli altri a sinistra, il costo esplode a $O(n)$.
- **La Trappola della Cancellazione nelle Liste:** Domanda classica: "*Cancellare un elemento da una lista doppiamente concatenata costa $O(n)$* ". **FALSO** (a metà). Se ti danno *già il puntatore* esatto al nodo da cancellare, basta scavalcarlo collegando il precedente al successivo: costo $O(1)$. Diventa $O(n)$ solo se prima devi *cercare* il valore scorrendo tutta la lista dalla testa.

2. ALBERI E ALBERI BINARI DI RICERCA (BST)

Un albero binario è una struttura dove ogni nodo ha al massimo due figli (Left e Right).

La Teoria (Zero Omissioni & Matematica Umana)

- **La Proprietà del BST:** In un Albero Binario di Ricerca (BST), tutto ciò che sta nel sottoalbero **sinistro** è $\leq$ del padre, e tutto ciò che sta nel sottoalbero **destro** è $\geq$ del padre.
- **Attraversamenti (Le Visite):** Ci sono tre modi per esplorare un albero, a seconda di quando "visiti" (stampi) il padre:
 - *PreOrder:* Padre, poi Sinistro, poi Destro.
 - *PostOrder:* Sinistro, poi Destro, poi Padre.
 - *InOrder:* Sinistro, poi Padre, poi Destro.
- **Matematica Umana (La ricerca):** Il tempo per cercare il Minimo (andare tutto a sinistra), il Massimo (andare tutto a destra) o un valore specifico, dipende dall'altezza dell'albero h . Quindi il costo è $O(h)$.

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto dell'InOrder:** "*L'attraversamento InOrder di un BST produce i nodi in ordine casuale*". **FALSO**. L'attraversamento InOrder di un BST stampa le chiavi in ordine **strettamente crescente**. Ricordalo come oro.
- **Il Trabocchetto dell'Altezza:** "*La ricerca in un BST costa sempre $O(\log n)$* ". **FALSO MORTALE**. Costa $O(h)$. Se l'albero è "bilanciato" allora $h = \log n$. Ma se inserisci numeri già ordinati (es. 1, 2, 3, 4), l'albero diventa una linea retta (lista), $h = n$, e la ricerca crolla a $O(n)$.

3. GLI ALBERI ROSSO-NERI (RB-Trees)

Poiché il BST normale può degenerare in $O(n)$, abbiamo inventato l'Albero Rosso-Nero: un BST che si "auto-bilancia" mantenendo sempre operazioni garantite in $O(\log n)$.

La Teoria (Zero Omissioni & Matematica Umana)

Per essere un RB-Tree, devono essere rispettate 4 (+1) regole ferree (il libro e gli appunti di Mattia ne fondono due, noi teniamo quelle canoniche per sicurezza):

1. Ogni nodo è **Rosso o Nero**.
 2. Ogni foglia (che nei RB sono nodi `NIL` sentinella) è **Nera**.
 3. *Regola Anti-Filotto*: I figli di un nodo Rosso **devono essere Neri**. È vietato avere due nodi rossi consecutivi.
 4. *Altezza Nera ($bh(x)$)*: Per ogni nodo, TUTTI i cammini che scendono verso le foglie devono incontrare **lo stesso numero di nodi neri**.
- **Matematica Umana (Il Teorema fondamentale)**: Grazie a queste regole, un albero RB non può mai "stirarsi" troppo da un solo lato. La matematica ci garantisce che l'altezza massima è: $h \leq 2 \log(n + 1)$. Tradotto in "Umano": l'altezza non supererà mai il doppio del logaritmo. Quindi le operazioni costeranno *sempre* e garantitamente $O(\log n)$.

Le Rotazioni e i Costi (FONDAMENTALE)

Quando inserisci un nodo, lo inserisci **Rosso** (per non sballare l'altezza nera). Ma se suo padre era rosso, violi la Regola 3. L'albero si aggiusta con ricolorazioni o **Rotazioni** (cambi di puntatori locali).

- *Costo di una singola rotazione*: Cambiare i puntatori costa un tempo istantaneo $O(1)$.

Lente per le Crocette (La Caccia al Trabocchetto)

Questo è il terreno di caccia preferito per i Vero/Falso.

- "L'inserimento in un RB-Tree può richiedere $\log n$ rotazioni". **FALSO**. Durante l'inserimento (`RB-Insert-Fixup`), si fanno al massimo **2 rotazioni**.
- "La cancellazione in un RB-Tree richiede un numero illimitato di rotazioni". **FALSO**. La cancellazione (`RB-Delete-Fixup`) si risolve con al massimo **3 rotazioni**.
- "È possibile unire due RB-Alberi in tempo logaritmico $O(\log n)$ ". **FALSO** (questa è apparsa nei tuoi esami passati!). L'operazione di fusione (Union) in tempo logaritmico è il superpotere degli *Heap Binomiali*, non degli Alberi Rosso-Neri!

4. AUMENTARE LE STRUTTURE DATI (Statistiche d'Ordine)

Cosa succede se vogliamo sapere in tempo record qual è il 7° numero più piccolo nell'albero? Un BST normale ci farebbe contare i nodi uno a uno ($O(n)$). Noi modifichiamo (aumentiamo) l'albero RB.

La Teoria (Zero Omissioni & Matematica Umana)

- Aggiungiamo a ogni nodo x un campo `x.size`: indica quanti nodi ci sono nel sottoalbero radicato in x (incluso sé stesso). La formula è: `x.size = x.left.size +`

`x.right.size + 1`.

- **OS-SELECT (Trova la i-esima posizione):** Se cerco il 17° elemento e il mio sottoalbero sinistro ha `size = 12`, so che la radice è il 13° elemento. Il 17° sarà sicuramente a destra, e dovrò cercare il 4° ($17 - 13 = 4$) elemento del sottoalbero destro. Costo: scendo lungo l'altezza, quindi $O(\log n)$.
- **OS-RANK (Trova il Rango di un nodo):** Ti do un puntatore a un nodo e voglio sapere in che posizione si classificherebbe se mettessi tutto in fila. Risalgo l'albero fino alla radice, sommando le `size` dei sottoalberi a sinistra. Costo: $O(\log n)$.
- **Il Mantenimento:** Se aggiungiamo `size`, ogni volta che facciamo un inserimento o una rotazione, i valori `size` si sballano! Nessun problema: le rotazioni sono locali (toccano solo 2 nodi alla volta) e ricalcolare `size` costa $O(1)$. Risalire verso l'alto per aggiornare i padri costa $O(\log n)$. Quindi l'albero "aumentato" non perde la sua efficienza originaria.

Lente per le Crocette (La Caccia al Trabocchetto)

- "In un RB-Tree è possibile mantenere in tempo logaritmico un campo che indica il numero di nodi dell'intero albero con la chiave minore alla chiave del nodo corrente". **VERA.** È l'esatta definizione dell'attributo `size` per calcolare il rango in $O(\log n)$! (Trovata nelle vecchie prove).
- "In un RB-Tree è possibile mantenere in tempo logaritmico un campo che indica il QUADRATO del numero di nodi nel sottoalbero". **FALSA.** (Apparsa anch'essa nei vecchi esami). Affinché un attributo possa essere mantenuto senza degradare le prestazioni dell'albero, esso deve poter essere ricalcolato in $O(1)$ *solo* a partire dalle informazioni dei figli immediati (come fa una semplice addizione `size = left + right + 1`). Mantenere proprietà globali o non lineari come i quadrati bloccherebbe il ripristino efficiente!

LENTE PER I PROBLEMI (Il Kit di Costruzione)

Nel maxi-problema della Seconda Parte, queste strutture sono i tuoi operai silenziosi.

- **Lo Scenario del Grafo (Stack e Queue):** Quando nei prossimi capitoli dovrai progettare algoritmi di visita dei Grafi (come esplorare un labirinto), la scelta della struttura cambierà tutto. Se nel tuo pseudocodice dici: "Prendo i nodi vicini e li metto in uno **Stack**", otterrai una visita in Profondità (DFS). Se dici: "Li metto in una **Coda (Queue)**", otterrai una visita in Ampiezza (BFS).
- **Lo Scenario "Lista dei Migliori in continua mutazione" (Alberi Aumentati):** Se il problema dice: "Ogni giorno entrano ed escono nuovi progetti dal sistema, e in qualsiasi momento il manager vuole sapere qual è il progetto alla mediana della classifica di redditività"...
 - Non puoi usare un array e ordinarlo ogni giorno ($O(n \log n)$ per ogni inserimento sarebbe troppo lento).

- Non puoi usare una normale Coda di Priorità (gli Heap sono bravi a darti il minimo/massimo, ma pietosi a darti la mediana).
- **La Scelta Vincente:** Scriverai *"Uso un Albero Rosso-Nero Aumentato con le Statistiche d'Ordine. In questo modo inserimenti e cancellazioni dei progetti costano $\Theta(\log n)$, e posso interrogare la mediana chiamando `OS-SELECT` con $i = n/2$ in tempo garantito $\Theta(\log n)$ ".* Hai appena risolto un problema difficilissimo in pochissime righe di progetto.



Prevenzione delle Sorprese (Oltre i PDF)

- **Puntatori Fantasma:** Attenzione se ti chiedono di calcolare il minimo in una lista concatenata non ordinata. Molti per distrazione dicono che basta leggere la coda o la testa, come negli Heap. No! Nelle liste non ordinate e non specializzate, trovare l'elemento minore costa SEMPRE il giro completo, quindi $\Theta(n)$.
- **Alberi Binari degeneri:** Per testare se hai capito che gli alberi non si bilanciano per magia, i prof potrebbero chiedere: *"In un Albero Binario di Ricerca (non RB) a cui inserisco i numeri $[100, 90, 80, 70]$, quanto costa cercare il 100?".* La risposta è $\Theta(1)$ (è rimasto alla radice), ma cercare il 70 costa $\Theta(n)$ perché l'albero si è sviluppato tutto come un "filotto" a sinistra!

CAPITOLO 7: PROGRAMMAZIONE DINAMICA (DP)

La Programmazione Dinamica è l'artiglieria pesante per i problemi di "Ottimizzazione" (trovare la soluzione *migliore*, la *più economica*, la *più veloce* tra tante possibili).

La Teoria (Zero Omissioni & Matematica Umana)

- **Differenza con il Divide et Impera:** Anche la DP spezza i problemi in sotto-problemi. MA, nel Divide et Impera i sotto-problemi sono *indipendenti* (es. Merge Sort divide a metà e non ci pensa più). Nella DP, i sotto-problemi sono **sovrapposti** (condividono a loro volta sotto-sotto-problemi). Ricalcolarli ogni volta farebbe esplodere il tempo a livelli esponenziali.
- **Il Segreto (Memoizzazione / Approccio Tabulare):** La DP calcola la soluzione di un sotto-problema *una volta sola* e la salva in una tabella (approccio Bottom-Up).
- **Le 4 Fasi dello Sviluppo:**
 1. Caratterizzare la *Sottostruttura Ottima* (dimostrare che la soluzione globale ottima contiene soluzioni ottime ai sotto-problemi).
 2. Definire l'equazione ricorsiva del valore ottimo.
 3. Calcolare il valore bottom-up (riempiendo la tabella).
 4. Ricostruire la soluzione ottima rileggendo la tabella.
- **Il Problema Base: Moltiplicazione di una Sequenza di Matrici:**

- L'ordine in cui moltiplichiamo le matrici cambia drasticamente il numero di calcoli da fare.
- *Matematica Umana*: Se ho 3 matrici, posso fare $(A_1 A_2) A_3$ oppure $A_1 (A_2 A_3)$. Se provo tutte le parentesi a forza bruta, il tempo è $\Omega(2^n)$ (inutile).
- *L'equazione Maledetta*: L'appunto mostra la formula del costo per moltiplicare da i a j :

$$m[i, j] = \min_{i \leq k < j} (m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j)$$

La Traduzione: Il costo per moltiplicare tutto da i a j è uguale al costo per moltiplicare la metà sinistra ($m[i, k]$), più il costo per la metà destra ($m[k + 1, j]$), più il costo "fisico" per unire questi due grossi blocchi di matrici. Come si calcola quest'ultimo? Moltiplicando le righe della prima per le colonne dell'ultima (i termini p_{i-1} , p_k , e p_j indicano proprio le dimensioni dei bordi delle matrici).

- *Complessità Finale*: Usando 3 cicli `for` annidati per riempire la tabella delle parentesi, il tempo crolla miracolosamente a $\Theta(n^3)$.

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto dell'Indipendenza**: "*La Programmazione Dinamica si applica a sotto-problemi indipendenti*". **FALSO**. Si applica a sotto-problemi sovrapposti.
- **Il Trabocchetto dello Zaino**: "*Il problema dello zaino è risolvibile con programmazione dinamica solo se per ogni sostanza si impone di usare tutto o niente (0/1)*". **VERA**. È apparsa nei tuoi esami. (Se lo zaino è frazionario, si usa l'algoritmo Greedy!).

Lente per i Problemi (Il Kit di Costruzione)

- Nella Seconda Parte, se ti chiedono di calcolare i cammini minimi fra *tutte* le coppie, o problemi di ottimizzazione complessa con vincoli temporali rigidi, la DP è la via. Il trucco è sempre definire la tabella: scrivi "*Uso la programmazione dinamica costruendo una matrice M in cui la cella i, j contiene il valore ottimo. Riempio la matrice in tempo $O(n^2)$ o $O(n^3)$* ".

Prevenzione delle Sorprese (La Tecnica "Taglia e Incolla")

- Come si dimostra la "Sottostruttura Ottima"? Il professore potrebbe chiederlo nei Vero/Falso. Si usa la logica del "Cut and Paste". *Umano*: Se la mia soluzione globale è ottima, ma al suo interno contiene un sotto-pezzo non ottimo, io posso "tagliare" via il pezzo scarso, "incollare" un pezzo migliore, e la soluzione globale diventerebbe ancora più ottima. Ma questo è un paradosso, perché era *già* ottima! Quindi, per forza, ogni sotto-pezzo deve essere ottimo.

CAPITOLO 8: ALGORITMI GOLOSI (GREEDY)

I cugini veloci ma "miopi" della Programmazione Dinamica.

La Teoria (Zero Omissioni & Matematica Umana)

- **Il Concetto:** Un algoritmo Greedy non fa calcoli complessi guardando al futuro. Fa la scelta che sembra *localmente la migliore in quell'esatto momento*, sperando che porti all'ottimo globale.
- **Il Problema Base: La Selezione di Attività:** Hai un'aula e 10 lezioni che vogliono usarla. Ogni lezione ha un inizio (s_i) e una fine (f_i). Come ne infili il maggior numero possibile senza sovrapposizioni?
 - *La Scelta Golosa (Greedy Choice):* Ordino le lezioni in base all'orario di fine. Scelgo sempre quella che **finisce per prima**. Perché? Perché libera la stanza il prima possibile per far entrare altri!
 - *La Regola di Compatibilità:* Una volta presa l'attività k , posso prendere la prossima attività m solo se il suo orario d'inizio è successivo alla fine della mia: $s_m \geq f_k$.
- **Complessità:** Se i dati sono già ordinati per orario di fine, basta scorrere l'array una volta: tempo $\Theta(n)$. Se devo ordinarli io prima, costa $O(n \log n)$. Può essere implementato in modo *ricorsivo* o *iterativo*, entrambi costano uguale.

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto dell'Infallibilità:** *"Un algoritmo goloso trova sempre la soluzione ottima globale"*. **FALSO**. Lo fa SOLO se il problema gode della "proprietà della scelta golosa" (ovvero se matematicamente il problema ammette che l'ottimo locale porti al globale, come nel caso delle attività o dell'algoritmo di Prim/Kruskal).
- **Il Trabocchetto delle Sottosequenze:** *"È possibile calcolare efficientemente sottosequenze massimali comuni di due sequenze mediante tecnica Greedy"*. **FALSA**. (Apparsa in esame). Quello è un problema classico di Programmazione Dinamica! Il Greedy sbaglierebbe.

Lente per i Problemi (Il Kit di Costruzione)

- Nei tuoi vecchi esami hai il problema del "labirinto con le monete" o dell'"auto ibrida/benzina". L'algoritmo di Dijkstra, che userai per risolverli (Capitolo 14), è un **Algoritmo Greedy!** Fa la scelta "golosa" di prendere sempre il nodo più vicino in quel momento. Quando dovrai *"argomentare circa la sua ottimalità"*, scriverai: *"L'algoritmo effettua scelte localmente ottime che garantiscono l'ottimo globale poiché i pesi degli archi non sono negativi, rispettando la proprietà della scelta golosa"*.

CAPITOLO 9: ALBERI BINOMIALI E HEAP BINOMIALI

Perché studiarli? Perché i normali Heap Binari (Capitolo 4) sono lenti a fare una cosa specifica: **Fondersi insieme** (costa $O(n)$). Gli Heap Binomiali fanno le fusioni in un lampo, $O(\log n)$.

La Teoria (Zero Omissioni & Matematica Umana)

- **L'Albero Binomiale (B_k):** È una struttura ricorsiva assurda. Un albero B_0 è un singolo nodo. Un B_k si forma prendendo DUE alberi B_{k-1} e attaccando la radice di uno come figlio (il più a sinistra) della radice dell'altro.
 - *Proprietà Fisse (Matematica Umana):*
 1. Ha 2^k nodi totali.
 2. La sua altezza è k .
 3. Quanti nodi ha alla profondità i ? Lo dice il coefficiente binomiale $\binom{k}{i}$ (ecco perché si chiamano così! Come il Triangolo di Tartaglia).
 4. Il grado massimo (quanti figli diretti ha la radice) è k .
- **L'Heap Binomiale:** È una foresta (un insieme) di Alberi Binomiali.
 - *Regola 1:* Ogni albero deve rispettare l'ordine min-heap (padre $\leq$ figli).
 - *Regola 2:* Non possono esserci due alberi B_k dello stesso grado (con lo stesso k).
 - *La Magia Binaria:* Quanti alberi ci sono in un Heap Binomiale di n nodi? Al massimo $\lfloor \log n \rfloor + 1$. In pratica, la struttura degli alberi ricalca i bit del numero in base 2. Se $n = 13$ (in binario 1101), l'Heap avrà un albero B_3 (8 nodi), un B_2 (4 nodi) e un B_0 (1 nodo). $8 + 4 + 1 = 13$.
- **Operazioni e Costi:**
 - *Minimo:* Si guardano solo le radici dei vari alberi (che sono al massimo $\log n$). Costo: $O(\log n)$.
 - *Unione (Il capolavoro):* Scandisce le liste delle radici. Se trova due B_k , li fonde in un B_{k+1} e procede. È esattamente come fare un'addizione binaria con il riporto! Costo: $O(\log n)$. Ci sono 4 casi tecnici nella procedura **UNION**, tutti volti a sistemare le radici con lo stesso grado tenendo in cima la chiave minore.
 - *Extract-Min:* Togli la radice minima, i suoi figli diventano un nuovo Heap Binomiale, e fai la *Unione* con il resto. Costo: $O(\log n)$.

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto dell'Unione:** "Non è possibile unire due heap binomiali in tempo logaritmico". **FALSA**. (Apparsa in esame). È letteralmente il loro scopo di vita. Lo fanno in $O(\log n)$.
- **Il Trabocchetto della Rimozione:** "Non è possibile rimuovere tutte le radici di uno heap binomiale in tempo $O(\log^2 n)$ ". **FALSA**. (Esame). Una singola estrazione della radice minima costa $\log n$. Se la ripeti per tutti gli elementi, il tempo sarà $n \log n$, figuriamoci se non si può fare per un subset!

- **Il Trabocchetto del Massimo:** *"La radice di un albero B_k ha grado k , che è il grado massimo di ogni nodo nell'albero". VERA.* È la proprietà 4 degli alberi binomiali.

Lente per i Problemi (Il Kit di Costruzione)

- Se un problema ti chiede di gestire infinite Code di Priorità che si devono fondere o "fagocitare" continuamente l'una con l'altra (es. fusioni di reti ospedaliere, accorpamento di branchi di nemici), il tuo mattone sarà: *"Istanzio un Heap Binomiale, poiché mi garantisce l'operazione di fusione di due cluster in tempo $\Theta(\log n)$ anziché il costo lineare degli heap binari standard".*

CAPITOLO 10: STRUTTURE DATI PER INSIEMI DISGIUNTI

Questa struttura (chiamata anche *Union-Find*) serve a gestire gruppi di elementi separati, senza alcuna sovrapposizione. Ogni gruppo è rappresentato da un "capo" (un elemento rappresentante scelto a caso all'interno del gruppo).

La Teoria (Zero Omissioni & Matematica Umana)

L'obiettivo è supportare tre operazioni base:

- **MAKE-SET(x)** : Crea un nuovo gruppo con il solo elemento x (che diventa il capo di sé stesso).
- **FIND-SET(x)** : Trova chi è il capo del gruppo a cui appartiene x .
- **UNION(x, y)** : Fonde i gruppi di x e y in un unico grande gruppo.
Variabili del gioco: n è il numero di **MAKE-SET** (gli elementi totali), m è il numero totale di operazioni miste effettuate. Ovviamente $m \geq n$, e le **UNION** possono essere al massimo $n - 1$.

Implementazione 1: Liste Concatenate

- Ogni gruppo è una lista. Il **head** punta al capo, il **tail** all'ultimo elemento. Ogni nodo ha un puntatore che torna dritto al capo.
- **MAKE-SET** e **FIND-SET** : Costano $O(1)$.
- **UNION** : Per unire due liste, attacchi la seconda alla fine della prima. Il problema è che devi aggiornare il puntatore al "capo" di *tutti* i nodi della seconda lista.
 - *Caso Pessimo:* Se unisci sempre una lista lunga a una corta e non stai attento, per aggiornare i puntatori farai $\Theta(n^2)$ operazioni.
 - *Euristica dell'unione pesata (Il Trucco):* Tieni traccia della lunghezza delle liste e attacchi SEMPRE la più corta alla più lunga. Così facendo, un elemento cambierà capo al massimo $\lfloor \log n \rfloor$ volte. Il tempo crolla a $O(m + n \log n)$.

Implementazione 2: Foreste di Insiemi Disgiunti (Il Capolavoro)

- Ogni gruppo è un albero. Ogni nodo punta *solo* a suo padre. La radice punta a sé stessa (ed è il capo).

- **UNION** unisce due alberi facendo puntare la radice di uno alla radice dell'altro.
- Per renderlo un capolavoro asintotico, si usano due euristiche combinate:
 1. *Unione per Rango*: Attacchi sempre la radice dell'albero più basso (rango minore) a quella dell'albero più alto. Eviti di creare alberi spilungoni.
 2. *Compressione del Cammino*: Quando fai **FIND-SET(x)** e risalì l'albero per trovare la radice, una volta trovata, stacchi tutti i nodi che hai attraversato e li fai puntare *direttamente* alla radice. L'albero si "schiaccia", diventando larghissimo e bassissimo.
- *Matematica Umana (La funzione α)*: Usando entrambe le euristiche, il tempo per m operazioni nel caso peggiore è $O(m\alpha(n))$. Cos'è $\alpha(n)$? È l'inversa della funzione di Ackermann. Cresce con una lentezza esasperante: per qualsiasi numero di input concepibile nell'universo, $\alpha(n) \leq 4$. Praticamente è una costante. Le operazioni costano *asintoticamente* $O(1)$.

Lente per le Crocette (La Caccia al Trabocchetto)

- **La Trappola della Complessità Liste**: *"L'operazione UNION in una lista concatenata costa sempre $O(1)$ ". **FALSA**. Collegare i puntatori finali costa $O(1)$, ma aggiornare il puntatore al rappresentante di ogni elemento trasferito costa $O(n)$.*
- **La Trappola della Compressione**: *"La compressione del cammino modifica il rango dei nodi". **FALSA**. Il rango è solo un limite superiore stimato dell'altezza, non l'altezza esatta, e la compressione del cammino schiaccia l'albero ma lascia i ranghi invariati.*

Lente per i Problemi (Il Kit di Costruzione)

- **Kruskal**: Quando arriveremo ai Grafi (Capitolo 13), l'algoritmo di Kruskal funziona magicamente *solo grazie* agli insiemi disgiunti. Se nel problema d'esame ti chiedono di calcolare un albero di copertura minimo (o gestire raggruppamenti di pixel/isole su una mappa), il tuo "mattoncino" sarà: *"Uso la foresta per insiemi disgiunti con euristiche di rango e compressione per verificare in tempo ammortizzato costante se due nodi appartengono già alla stessa componente connessa"*.

CAPITOLO 11: HASHING

L'Hashing serve a implementare i Dizionari (cercare, inserire, cancellare elementi) in tempo medio $O(1)$.

La Teoria (Zero Omissioni & Matematica Umana)

- **Indirizzamento Diretto**: Se l'universo delle chiavi U (tutti i possibili numeri di matricola) è piccolo, creo un array grande $|U|$ e metto l'elemento k nella cella k . Costo $O(1)$ garantito.

- *Problema:* Se U è enorme, la memoria esplode. E se inserisco solo K chiavi (con $K \ll U$), spreco il 99% dello spazio.
- **La Tavola Hash:** Risolve lo spreco. Uso un array di grandezza m molto più piccola di U . Uso una **funzione hash** $h(k)$ che prende la chiave e la trasforma in un indice tra 0 e $m - 1$.
 - *Collisioni:* Poiché ci sono più chiavi che celle, due chiavi diverse finiranno prima o poi nella stessa cella ($h(k_1) = h(k_2)$). Bisogna gestirle.
- **Il Fattore di Carico (α):** Si calcola come $\alpha = n/m$ (n elementi divisi per m celle). È il numero medio di elementi per cella.

Risoluzione delle Collisioni 1: Concatenamento (Chaining)

- Se due chiavi finiscono nella stessa cella, si "appendono" l'una all'altra formando una lista concatenata.
- *Costi e Matematica:* L'inserimento costa $O(1)$ (aggiungi in testa). La ricerca costa in media $\Theta(1 + \alpha)$. Se l'hashing è buono (uniforme), le liste restano corte.
- *Caso Pessimo:* Tutte le n chiavi finiscono per sfortuna nella stessa cella. La tavola diventa un'unica lista lineare. Ricerca: $\Theta(n)$.

Le Funzioni Hash

- **Divisione:** $h(k) = k \bmod m$ (il resto della divisione). *Regola d'oro:* mai scegliere m come potenza di 2 (es. 2^p), altrimenti la funzione hash ignorerà la maggior parte della chiave e guarderà solo gli ultimi p bit. Si sceglie un numero primo lontano da potenze di 2.
- **Moltiplicazione:** $h(k) = \lfloor m(kA \bmod 1) \rfloor$. *Matematica Umana:* Prendi la chiave k , la moltiplichi per un decimale fisso A (tra 0 e 1). Tieni solo la parte dopo la virgola ($\bmod 1$) e la moltiplichi per l'ampiezza della tavola m . Il vantaggio è che m può tranquillamente essere una potenza di 2.

Risoluzione delle Collisioni 2: Indirizzamento Aperto

- NON si usano liste. Tutti i dati stanno *dentro* la tabella. Se la cella è occupata, vai a cercare la prossima cella libera seguendo una "sequenza di ispezione".
- *Matematica:* Il fattore di carico α non può MAI superare 1 (la tavola al massimo si riempie del tutto).
- *I Tre Metodi di Ispezione:*
 1. *Lineare:* Cella occupata? Vai a quella dopo (+1), poi quella dopo, ecc. Soffre di **Addensamento Primario** (si formano lunghi blocchi di celle occupate e perdi un mucchio di tempo a scavalcarli).

2. *Quadratica*: Cella occupata? Fai salti crescenti al quadrato (+1, +4, +9...). Migliore, ma soffre di Addensamento Secondario.
3. *Doppio Hashing (Il Migliore)*: Cella occupata? Usi una SECONDA funzione hash $h_2(k)$ per decidere l'ampiezza del salto. Le sequenze sembrano casuali.
- *Cancellazione*: È difficilissima. Non puoi semplicemente svuotare la cella (mettere **NIL**), altrimenti spezzaresti la sequenza di ricerca per elementi inseriti successivamente e l'algoritmo non li troverebbe più. Si usa una speciale etichetta "DELETED".

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto della Cancellazione**: "Nell'indirizzamento aperto, l'operazione di cancellazione si esegue sostituendo l'elemento con NIL". **FALSISSIMA**. Come appena detto, rompe la catena di ispezione.
- **La Trappola del Fattore di Carico**: "In una tavola hash con concatenamento, il fattore di carico non può superare 1". **FALSA**. Col concatenamento (liste) puoi avere 100 elementi in 10 celle ($\alpha = 10$). È nell'indirizzamento aperto che α è vincolato a ≤ 1 .
- **Il Vincolo del Doppio Hashing**: "Nel doppio hashing, la seconda funzione hash $h_2(k)$ può restituire qualsiasi valore". **FALSO**. $h_2(k)$ DEVE essere relativamente primo rispetto a m (non avere divisori comuni), altrimenti la tua ispezione tornerà su sé stessa in un loop chiuso, ignorando intere celle vuote della tabella.

Lente per i Problemi (Il Kit di Costruzione)

- L'Hashing non genera quasi mai un maxi-problema da 30 punti da solo, ma è l'ottimizzatore perfetto. Se nel testo del problema ti chiedono di memorizzare indirizzi IP o stringhe testuali in modo da poter verificare istantaneamente se le hai già incontrate, scriverai: "Poiché l'universo delle chiavi è vasto, uso una Tavola Hash. In questo modo le operazioni di check (SEARCH) richiederanno tempo atteso $\Theta(1)$, mantenendo la complessità globale del mio algoritmo ottima".

Prevenzione delle Sorprese (Oltre i PDF)

- Cosa succede se la tavola ad Indirizzamento Aperto si riempie totalmente ($\alpha = 1$)? Questa è la classica domanda spietata da fine esame. La risposta è l'**Hashing Dinamico**: si alloca una nuova tavola hash di dimensione doppia ($2m$), si sceglie una nuova funzione hash, e si "ri-hashano" (Rehash) tutti gli elementi vecchi nella nuova tavola. Il costo di questa singola operazione è altissimo $\Theta(n)$, ma spalmato (ammortizzato) su tutte le operazioni precedenti torna a essere in media $O(1)$.

1. RAPPRESENTAZIONE DEI GRAFI

Un grafo $G = (V, E)$ è un insieme di Vertici (V , i nodi) e di Archi (E , i collegamenti). Per darli in pasto a un computer, ci sono due modi principali:

La Teoria (Zero Omissioni & Matematica Umana)

- **Liste di Adiacenza:** Per ogni vertice, creo una lista che contiene solo i "vicini" a cui è direttamente collegato.
 - *Memoria (Spazio):* Occupa $\Theta(V + E)$. Se il grafo non è orientato, ogni arco (u, v) compare sia nella lista di u che in quella di v , quindi la somma delle lunghezze delle liste è $2|E|$. Se è orientato, è esattamente $|E|$.
 - *Scopo:* Perfetto per i grafi "sparsi" (pochi archi rispetto al massimo possibile).
- **Matrice di Adiacenza:** Una griglia quadrata $V \times V$. Se c'è un arco da i a j , metto 1 nella cella (i, j) , altrimenti metto 0.
 - *Memoria (Spazio):* Occupa sempre e comunque $\Theta(V^2)$, a prescindere che ci siano zero archi o un milione.
 - *Scopo:* Perfetto per i grafi "densi" (tanti archi) o per sapere all'istante (in tempo $O(1)$) se due nodi specifici sono collegati.

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto dell'Applicabilità:** *"Le matrici di adiacenza possono essere usate solo per rappresentare grafi completi"*. **FALSO** (apparsa in esame!). Le matrici *possono* rappresentare qualsiasi grafo, semplicemente sprecano un mare di memoria se il grafo ha pochi archi.
- **Il Trabocchetto della Simmetria:** *"Se un grafo è orientato, allora la sua matrice di adiacenza è simmetrica"*. **FALSO**. È nei grafi *NON orientati* che la matrice è perfettamente speculare rispetto alla diagonale ($A = A^T$).
- **La Verifica di Connessione:** *"Un grafo è rappresentabile con liste di adiacenza solo se è connesso"*. **FALSA**. Nessun limite del genere esiste.

2. VISITA IN AMPIEZZA (BFS - Breadth-First Search)

L'algoritmo che si espande come i cerchi nell'acqua quando lanci un sasso.

La Teoria (Zero Omissioni & Matematica Umana)

- **Come funziona:** Partendo da una sorgente s , esplora prima tutti i nodi a distanza 1, poi quelli a distanza 2, e così via.
- **I Colori:**
 - *Bianco:* Mai visto.
 - *Grigio:* Scoperto, ma i suoi vicini devono ancora essere ispezionati. Forma la "frontiera".
 - *Nero:* Ispezionato totalmente.
- **La Struttura Dati:** Usa una **Coda (Queue) FIFO** per gestire i nodi Grigi.

- **Il Tracciamento:** Per ogni nodo calcola $d[u]$ (la distanza dalla sorgente) e $\pi[u]$ (il nodo "padre" da cui sono arrivato). Il sottografo dei padri forma un Albero (Albero BFS).
- **Complessità:** $O(V + E)$. Perché? Per inizializzare perdo $O(V)$, e poi guardo ogni arco una sola volta (o due se non orientato), impiegando $O(E)$.

Lente per i Problemi (Il Kit di Costruzione)

- **Lo Scenario Vincente:** *"Trova il percorso con il minor numero di stanze tra l'entrata e l'uscita del labirinto".* Quando un problema chiede un "cammino minimo" in un grafo in cui gli archi **non hanno pesi** (o costano tutti esattamente 1), la BFS è l'arma definitiva.
- **La Risposta nel Compito:** *"Rappresento il labirinto come un grafo non pesato. Applico la BFS partendo dall'entrata. Poiché la BFS garantisce la scoperta dei nodi per livelli di distanza crescente, il percorso trovato seguendo i puntatori π dall'uscita a ritroso sarà garantitamente il più breve in termini di passi. Complessità: $O(V + E)$ ".*

3. VISITA IN PROFONDITÀ (DFS - Depth-First Search)

L'algoritmo dell'esploratore solitario in un labirinto: vai sempre dritto finché non sbatti contro un muro, poi torni indietro al primo bivio.

La Teoria (Zero Omissioni & Matematica Umana)

- **Come funziona:** A differenza della BFS, è ricorsivo (usa lo Stack). Va più a fondo possibile prima di tornare indietro (backtrack).
- **Foresta DF:** Se il grafo è disconnesso o la sorgente iniziale non raggiunge tutto, la DFS riparte da un altro nodo bianco a caso, generando non un solo albero, ma una *Foresta* di alberi.
- **I Tempi (Timestamps):** Alla DFS non interessa la "distanza", ma il tempo. Ogni nodo ha:
 - $d[u]$: Tempo di scoperta (quando diventa grigio).
 - $f[u]$: Tempo di fine (quando diventa nero, cioè ho ispezionato tutti i suoi vicini).
- **Teorema delle Parentesi:** Fondamentale. I tempi di scoperta e fine sono come le parentesi $()$ in un'equazione. Per due nodi u e v , i loro intervalli temporali o sono completamente disgiunti, oppure uno è interamente dentro l'altro (il che significa che uno è discendente dell'altro).
- **Teorema del Cammino Bianco:** Un nodo v diventerà discendente di u se, nel momento esatto in cui scopro u , esiste un percorso fatto solo di nodi bianchi per arrivare a v .
- **Complessità:** $\Theta(V + E)$.

La Classificazione degli Archi (CRITICO!)

La DFS "smaschera" la vera natura degli archi nel grafo:

1. *Archi d'albero*: Vai verso un nodo Bianco.
2. *Archi all'indietro (Back Edges)*: Vai verso un nodo Grigio (stai tornando a un antenato ancora aperto). **Rivelano la presenza di un CICLO.**
3. *Archi in avanti*: Vai verso un nodo Nero discendente ($d[u] < d[v]$).
4. *Archi trasversali*: Vai verso un nodo Nero non imparentato ($d[u] > d[v]$).

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto dell'Albero vs Foresta**: "La BFS genera una foresta, la DFS un albero". **FALSO**. È il contrario: la DFS genera una Foresta (DF Forest) perché viene richiamata esplicitamente dal ciclo principale su tutti i nodi bianchi rimasti, la BFS classica genera un solo Albero partendo da una singola sorgente.

4. ORDINAMENTO TOPOLOGICO

Hai una serie di compiti in cui alcuni devono essere fatti prima di altri (es. infilarsi le mutande prima dei pantaloni, o il classico problema di determinare l'ordine degli esami propedeutici).

La Teoria (Zero Omissioni & Matematica Umana)

- **Il Requisito (Condizione Necessaria)**: Funziona SOLO su **DAG** (Directed Acyclic Graph), ovvero grafi orientati senza cicli. (Se per fare Analisi 2 serve Analisi 1, e per fare Analisi 1 serve Analisi 2, il sistema va in stallo).
- **L'Algoritmo**:
 1. Lancia la DFS sul grafo.
 2. Ogni volta che la DFS ha *finito* un nodo (quando lo colora di Nero e gli assegna $f[u]$), infila quel nodo in **testa** a una lista concatenata (quindi in ordine decrescente rispetto ai tempi di fine).
 3. Restituisci la lista.
- **Complessità**: $\Theta(V + E)$ (esattamente il costo della DFS, più $O(1)$ per ogni inserimento in testa alla lista).

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto del Ciclo**: "È possibile verificare se un grafo orientato è aciclico in $\Theta(V + E)$ ". **VERA**. È sufficiente lanciare una DFS e controllare se viene trovato un "Arco all'indietro" verso un nodo Grigio. Se sì, c'è un ciclo, altrimenti è aciclico.
- **L'Ordine Sbagliato**: Spesso chiedono se l'ordinamento topologico ordina in base al tempo di *scoperta* $d[u]$. **FALSO**. Lo ordina al contrario in base al tempo di *completamento* $f[u]$.

5. COMPONENTI FORTEMENTE CONNESSE (SCC)

In un grafo orientato, una Componente Fortemente Connessa (SCC) è un sottoinsieme di nodi che formano un "club chiuso" in cui **tutti possono raggiungere tutti**.

La Teoria (Zero Omissioni & Matematica Umana)

- L'algoritmo di Kosaraju sfrutta una magia: se io capovolgo tutti i sensi unici del grafo (creando il grafo trasposto G^T), i "club chiusi" (le SCC) non cambiano! Se io posso andare da Roma a Milano e da Milano a Roma, invertendo le strade potrò comunque farlo.
- **L'Algoritmo (in 4 passi):**
 1. Lancia la DFS su G per calcolare i tempi di fine $f[u]$ di tutti i nodi.
 2. Calcola G^T invertendo tutti gli archi.
 3. Lancia una seconda DFS su G^T , ma **forzando il ciclo principale a pescare i nodi in ordine decrescente del loro tempo di fine $f[u]$** (calcolato al passo 1).
 4. Ogni albero DF generato nella seconda visita è una Componente Fortemente Connessa.
- **Complessità:** $\Theta(V + E)$ (due visite DFS e una trasposizione degli archi, che costa sempre un tempo lineare).

Lente per le Crocette (La Caccia al Trabocchetto)

- **La Trappola della Complessità:** *"Il calcolo delle componenti fortemente connesse di un grafo appartiene a $O(V^2 + E)$ ".* **FALSO** (o quantomeno inutilmente largo rispetto al limite asintotico reale). La risposta esatta è $\Theta(V + E)$.
- **Componenti Connesse vs Fortemente Connesse:** *"In un grafo non orientato le componenti fortemente connesse non sono più numerose delle componenti connesse".* **VERA**. In un grafo NON orientato, la "forte" connessione non ha senso logico, perché tutte le strade sono a doppio senso, quindi una componente connessa coincide esattamente con una fortemente connessa.

Prevenzione delle Sorprese (Oltre i PDF)

- **Il Grafo delle Componenti:** I prof amano testare un concetto teorico profondo: se io prendo un grafo complesso, "schiaccio" ogni intera Componente Fortemente Connessa riducendola a un singolo macro-nodo (chiamato Grafo delle Componenti G^{SCC}), che tipo di grafo ottengo? La risposta matematica infallibile è: **Ottieni SEMPRE un DAG (Directed Acyclic Graph)**. Non può mai avere cicli, perché se due macro-nodi formassero un ciclo, per definizione dovrebbero fondersi in un'unica gigantesca SCC!

CAPITOLO 13: ALBERI DI CONNESSIONE MINIMI (MST)

La Teoria (Zero Omissioni & Matematica Umana)

Il problema: hai un grafo **connesso e non orientato** in cui gli archi hanno dei "costi" (pesi). Vuoi collegare TUTTI i nodi tra loro spendendo il minimo possibile.

- **Perché "Albero"?** Perché se colleghi tutti i nodi col minimo costo, non puoi avere cicli (se hai un ciclo, puoi togliere un arco e risparmiare, restando comunque collegato). Un albero che copre $|V|$ vertici avrà sempre e matematicamente $|V| - 1$ archi.
- **Il Metodo Generico e l'Arco Sicuro:** Si parte con un insieme vuoto e si aggiunge un arco alla volta. L'arco che si aggiunge deve essere un "Arco Sicuro".
- **Il Teorema del Taglio (Matematica Umana):** Immagina di tagliare il grafo in due gruppi di nodi (Taglio S e $V - S$). Gli archi che fanno da ponte tra i due gruppi si dicono che "attraversano il taglio". Se prendi l'arco che costa di meno tra quelli che attraversano il taglio (l'**arco leggero**), la matematica garantisce che quell'arco appartiene sicuramente alla soluzione ottima.

1. Algoritmo di Kruskal (Il Costruttore di Foreste)

- *L'Idea:* È un algoritmo Greedy. Ordina tutti gli archi dal più economico al più costoso. Poi li scorre: se l'arco unisce due nodi che sono già collegati tra loro (creando un ciclo), lo scarta. Altrimenti lo aggiunge.
- *Il Motore Nascosto:* Come fa a sapere istantaneamente se due nodi sono già collegati? Usa la struttura **Union-Find (Insiemi Disgiunti)** del Capitolo 10.
- *Complessità:* Ordinare gli archi costa $O(E \log E)$. Le operazioni Union-Find costano $O(E \alpha(V))$. Il tempo è dominato dall'ordinamento. *Matematica Umana:* Poiché in un grafo il numero massimo di archi E è V^2 , il logaritmo $\log(V^2)$ equivale a $2 \log V$. Quindi asintoticamente $O(E \log E)$ è identico a $O(E \log V)$.

2. Algoritmo di Prim (La Macchia d'Olio)

- *L'Idea:* Parte da un nodo a caso (la radice) e fa crescere un singolo albero, aggiungendo a ogni passo il nodo esterno più "economico" da raggiungere dall'albero.
- *Il Motore Nascosto:* Usa una **Coda di Min-Priorità (Min-Heap)** per trovare subito il nodo col peso minore.
- *Complessità:* Dipende dall'implementazione della coda.
 - Con array lineare: $O(V^2)$.
 - Con Min-Heap binario: $O(E \log V)$ (Ideale per grafi sparsi).
 - Con Heap di Fibonacci: $O(E + V \log V)$ (Il più veloce in assoluto).

Lente per le Crocette (La Caccia al Trabocchetto)

- **Il Trabocchetto dei Pesi Negativi:** *"Gli algoritmi di Kruskal e Prim non funzionano se il grafo ha pesi negativi"*. **FALSO!** Funzionano perfettamente. È Dijkstra (sui cammini minimi) a fallire con i pesi negativi, non gli MST.
- **La Topologia Esatta:** *"Kruskal mantiene sempre un singolo albero in crescita"*. **FALSO.** Kruskal unisce alberi sparsi formando una *Foresta* che diventa un unico albero solo alla fine. È Prim che mantiene sempre e solo un singolo albero in crescita.
- **L'Albero dei Cammini Minimi:** *"In un grafo connesso un albero dei cammini minimi è anche un albero di copertura, sebbene non necessariamente di costo minimo"*. **VERA.** È un albero e tocca tutti i nodi (quindi li "copre"), ma ottimizza le distanze dalla sorgente, non il "peso totale dei cavi" usati.

CAPITOLO 14: CAMMINI MINIMI DA SORGENTE UNICA (SSSP)

La Teoria (Zero Omissioni & Matematica Umana)

Il problema: dato un nodo di partenza s in un grafo **orientato e pesato**, trovare il percorso più economico per raggiungere tutti gli altri nodi v . Il "costo" $\delta(s, v)$ è la somma dei pesi degli archi.

- **Proprietà del Sottocammino:** Se il percorso migliore da Milano a Roma passa per Firenze, allora il pezzo di percorso fino a Firenze DEVE essere il percorso migliore da Milano a Firenze. Non si scappa.
- **I Pesi Negativi e il Limite Fisico:** Se c'è un "ciclo di peso negativo" (un anello che girandoci dentro ti fa *guadagnare* all'infinito abbassando il costo), il cammino minimo diventa matematicamente $-\infty$. Un cammino minimo non può *mai* contenere un ciclo, quindi al massimo visiterà $|V|$ vertici usando al massimo $|V| - 1$ archi.
- **Il Rilassamento (L'Operazione Base):** È la formula magica di questo capitolo. Se la stima attuale per arrivare a v è peggiore della stima per arrivare a u più l'arco da u a v , allora aggiorno la stima: `if (v.d > u.d + w(u,v)) { v.d = u.d + w(u,v); v.pi = u }`.

1. Algoritmo di Dijkstra (Il Greedy Ottimista)

- *L'Idea:* Parto dalla sorgente. Uso una coda di Min-Priorità per pescare il nodo u "più vicino". Lo dichiaro chiuso (lo metto in S) e "rilasso" tutti i suoi vicini. Ripeto finché la coda è vuota.
- *Condizione Invalicabile:* **Nessun arco può essere negativo.** Se ce n'è anche solo uno, Dijkstra va in panico e fallisce.
- *Complessità:* Esattamente la stessa di Prim. Con Min-Heap è $O(E \log V)$.

2. Algoritmo di Bellman-Ford (Il Pessimista Metodico)

- *L'Idea:* Non fa scelte furbe. Prende TUTTI gli archi del grafo e li "rilassa" bloccando tutto per $|V| - 1$ volte.

- *Perché proprio $|V| - 1$?* Perché, come detto prima, un cammino minimo ha al massimo $|V| - 1$ archi. Ripetendo l'operazione, la "verità" si propaga dal primo arco fino all'ultimo.
- *La caccia al ciclo negativo:* Finito il ciclo, fa un ultimo giro. Se un arco può essere ancora rilassato, significa che c'è un ciclo negativo! Ritorna FALSE.
- *Complessità:* Fissa a $O(V \cdot E)$.

3. DAG-SP (Cammini Minimi su DAG)

- *L'Idea:* Se il grafo orientato NON ha cicli (DAG), c'è una scorciatoia assurda. Lo ordini topologicamente (Cap 12) e rilassi i nodi in quell'esatto ordine, muovendoti da sinistra a destra.
- *Potere Nascosto:* Può avere pesi negativi e non fallirà mai (non essendoci cicli, non ci possono essere cicli negativi).
- *Complessità:* Istantaneo. Tempo $\Theta(V + E)$.

Lente per le Crocette (La Caccia al Trabocchetto)

- **Dijkstra e Cicli Negativi:** *"L'algoritmo di Dijkstra per cammini minimi è applicabile solo se non esistono cicli negativi".* **FALSA** (Questa è apparsa uguale, attento al trabocchetto semantico!). Dijkstra fallisce appena c'è un *arco* di peso negativo, non ha bisogno che ci sia un *ciclo* negativo per sbagliare!
- **Bellman-Ford Termina Subito:** *"Bellman-Ford termina sempre in $\Theta(V \cdot E)$ ".* **VERA.** Rilassa in ogni caso tutto $V - 1$ volte.

Lente per i Problemi (Il Kit di Costruzione)

Questo è il tuo arsenale per la "Seconda Parte" dell'esame.

- **Problema della "Gara Podistica" / "Auto ibrida"** (dai tuoi esami).
 - *Lo scenario:* Hai nodi, percorsi e consumi che dipendono da "salite/discese".
 - *La mossa:* Trasformi il problema in un grafo orientato. I consumi diventano i *pesi* degli archi.
 - *La Scelta Vincente:* Se i consumi (pesi) sono tutti ≥ 0 , scriverai: *"Modello con grafo orientato pesato. Uso l'algoritmo di Dijkstra che mi garantisce la soluzione ottima e il percorso in $O(E \log V)$ ".*
 - *L'Ottimizzazione Critica:* Se l'auto "recupera energia" in discesa, significa che il peso dell'arco diventa *negativo*. Dijkstra esploderebbe. La tua risposta da 30 e lode sarà: *"Poiché sono presenti pesi negativi dovuti al recupero energia, Dijkstra non è applicabile. Se il tracciato non ha anelli chiusi in discesa (nessun ciclo negativo), applico l'algoritmo di **Bellman-Ford** in $O(V \cdot E)$ ".*
- **Problema dei "Progetti da completare in n mesi":**

- *La mossa*: Spesso questi problemi generano dipendenze lineari (questo prima di quello). Se è un DAG, si userà **DAG-SP**. Se ti chiedono il percorso PIÙ LUNGO (cammino critico), scriverai: *"Inverto il segno di tutti i pesi e lancio DAG-SP in $\Theta(V + E)$, cosa consentita dall' algoritmo che tollera i pesi negativi sui DAG".* (È menzionato esplicitamente negli appunti)!

CAPITOLO 15

1. IL PROBLEMA E L'APPROCCIO NAIVE

La Teoria (Zero Omissioni & Matematica Umana)

- L'obiettivo è creare una matrice $D = (d_{ij})$ in cui ogni cella contiene il costo del cammino minimo per andare dal vertice i al vertice j . L'input è la matrice di adiacenza dei pesi W .
- *L'Approccio Naive (Forza Bruta)*: Lanciamo gli algoritmi del Capitolo 14 (Dijkstra o Bellman-Ford) ripetendoli per $|V|$ volte, cioè usandoli una volta per ogni singolo vertice come sorgente.
 - *Con Dijkstra (niente pesi negativi)*: Ripetuto V volte costa $O(V^3)$ con array lineare, oppure $O(VE \log V)$ usando un Min-Heap.
 - *Con Bellman-Ford (pesi negativi ammessi)*: Ripetuto V volte costa $O(V^2E)$. Se il grafo è denso ($E \approx V^2$), il tempo esplode al catastrofico limite di $O(V^4)$.

2. CAMMINI MINIMI E MOLTIPLICAZIONE DI MATRICI

Si può risolvere il problema usando la Programmazione Dinamica (DP), notando un'incredibile somiglianza tra i cammini minimi e la moltiplicazione tra matrici.

La Teoria (Zero Omissioni & Matematica Umana)

- **L'Equazione Ricorsiva**: Sia $l_{ij}^{(m)}$ il costo del cammino minimo da i a j usando al massimo m archi. Per calcolare il percorso con m archi, prendo il percorso migliore con $m - 1$ archi fino a un nodo intermedio k , e ci aggiungo l'ultimo arco da k a j .
 - *Formula Umana*: $l_{ij}^{(m)} = \min(l_{ik}^{(m-1)} + w_{kj})$ esplorando tutti i possibili k .
- **Il Parallelo Matematico**: Questa formula è identica al prodotto riga per colonna tra matrici, basta sostituire il "+" con la funzione "minimo", e la "moltiplicazione" con la somma "+".
- **L'Ottimizzazione (Repeated Squaring)**: Partiamo da $L^{(1)} = W$. Invece di calcolare la matrice passo passo fino a $L^{(n-1)}$ (che costerebbe $\Theta(n^4)$), "raddoppiamo" ogni volta. Calcoliamo W^2 , poi W^4 , poi W^8 e così via.
 - *Costo Finale*: Raddoppiando, facciamo solo $\log n$ prodotti tra matrici. Ognuno costa n^3 . Il tempo di questo approccio è $\Theta(n^3 \log n)$.

3. ALGORITMO DI FLOYD-WARSHALL

È l'algoritmo definitivo e più elegante per risolvere il problema sui grafi densi. Usa la Programmazione Dinamica ma in modo diverso dalla moltiplicazione di matrici.

La Teoria (Zero Omissioni & Matematica Umana)

- L'algoritmo si basa sui **vertici intermedi**. Un vertice intermedio in un percorso $A \rightarrow B \rightarrow C \rightarrow D$ è un qualsiasi vertice che non sia l'inizio o la fine (in questo caso B e C).
- **La Matrice $D^{(k)}$** : Immagina di costruire una matrice in cui ti è permesso usare *solo* i vertici da 1 a k come tappe intermedie.
- **L'Equazione Logica**: Quando sblocchi la possibilità di usare il vertice k , per ogni coppia (i, j) ti fai una singola domanda: *"Mi conviene andare direttamente da i a j (usando le regole vecchie), oppure mi costa meno andare da i a k , e poi da k a j ?"*.
 - Formula: $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$.
- **Complessità**: L'algoritmo usa 3 cicli `for` annidati (scorre k , scorre i , scorre j). Il tempo è un macigno fisso: $\Theta(V^3)$.

4. ALGORITMO DI JOHNSON (Per Grafi Sparsi)

È il capolavoro assoluto del corso. Permette di usare il velocissimo Dijkstra su *tutti* i nodi, anche se ci sono pesi negativi!

La Teoria (Zero Omissioni & Matematica Umana)

- Se il grafo è sparso, chiamare Dijkstra V volte costerebbe $O(VE \log V)$, che è molto meglio del $\Theta(V^3)$ di Floyd-Warshall. Ma se ci sono archi negativi Dijkstra esplode.
- **Il Trucco (Ricalcolo dei Pesi / Reweighting)**: Johnson inventa una formula magica per cambiare i pesi di tutto il grafo in modo che diventino tutti ≥ 0 , **senza alterare la classifica dei percorsi più brevi**.
- *L'Algoritmo in 4 Passi*:
 1. Aggiunge un nodo finto s collegato a tutti gli altri nodi con archi di peso 0.
 2. Lancia **Bellman-Ford** partendo da s . Se Bellman-Ford trova un ciclo negativo, l'algoritmo si ferma e dichiara il fallimento. Altrimenti, calcola il costo per arrivare da s ad ogni nodo v , e lo chiama $h(v)$.
 3. *La Formula del Ricalcolo*: Applica la magia: $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$. In questo modo, matematicamente, nessun arco sarà più negativo.
 4. Rimuove s e, dato che ora tutti i pesi sono positivi, lancia **Dijkstra** per ogni singolo vertice del grafo. Alla fine "sottrae" il trucco per restituire i pesi originali.

- **Complessità:** Bellman-Ford costa $O(VE)$. Lanciare Dijkstra V volte con Min-Heap costa $O(VE \log V)$. Il totale è dominato da Dijkstra: $O(V^2 \log V + VE)$.

Lente per le Crocette (La Caccia al Trabocchetto)

Questo capitolo è una miniera d'oro per chi scrive i quiz.

- **L'illusione di Floyd-Warshall:** *"La complessità di Floyd-Warshall è $\Theta(V^3)$ ".* **VERA.** *"La complessità di Floyd-Warshall è $\Omega(V^2)$ ".* **VERA.** Fai molta attenzione: se una cosa è strettamente uguale a un cubo, è anche "almeno" uguale a un quadrato (matematica asintotica del Cap 2). L'hai sbagliata in una simulazione!
- **Il Raggio d'Azione di Johnson:** *"L'algoritmo di Johnson produce risposte corrette SOLO se applicato a grafi sparsi".* **FALSA** (È apparsa due volte nei tuoi PDF!). Johnson funziona perfettamente su *qualsiasi* grafo. È "particolarmente indicato" per i grafi sparsi perché è lì che batte Floyd-Warshall in velocità, ma è corretto su tutti!
- **Pesi negativi vs Cicli negativi:** *"L'algoritmo di Johnson non funziona quando esistono archi negativi".* **FALSA.** È stato creato *esattamente* per gestire gli archi negativi. Fallisce, come tutti, solo se ci sono *cicli* negativi (che rileva grazie a Bellman-Ford interno).

Lente per i Problemi (Il Kit di Costruzione)

Nei maxi-problemi, le tabelle di routing globale sono il pane quotidiano.

- **Scenario A:** Hai una rete aziendale o stradale e devi calcolare *tutte* le combinazioni di distanze, e la rete è densa (tutti collegati a quasi tutti).
 - *Soluzione:* Scriverai: *"Poiché il grafo è denso, uso l'algoritmo di Floyd-Warshall in $\Theta(V^3)$ per calcolare la matrice dei cammini minimi"*
- **Scenario B:** Hai una rete logistica di consegne molto ampia ma "sparsa" (ogni città è collegata solo ad altre 2 o 3, non a tutte). Ci sono bonus e malus (pesi positivi e negativi).
 - *Soluzione:* Scriverai: *"Essendo il grafo sparso e presentando archi di peso negativo, Floyd-Warshall in V^3 sarebbe troppo lento. Uso l'algoritmo di Johnson, che tramite ricalcolo dei pesi mi garantisce i percorsi in $O(V^2 \log V + VE)$ ".* Dimostrerai una padronanza totale della scelta strutturale.

Prevenzione delle Sorprese (Oltre i PDF)

- **Perché la magia di Johnson funziona? (La Proprietà Telescopica):** Per smascherarti all'orale o in una crocetta difficile, potrebbero chiederti *perché* aggiungere $+h(u) - h(v)$ non sbaglia le classifiche dei percorsi.
 - *La Spiegazione Umana e Logica:* Se tu percorri un cammino da $A \rightarrow B \rightarrow C \rightarrow D$, e calcoli il nuovo peso totale, avrai: $(W_{AB} + h(A) - h(B)) +$

$$(W_{BC} + h(B) - h(C)) + (W_{CD} + h(C) - h(D)).$$

- Guarda i termini centrali! $-h(B)$ si cancella con $+h(B)$. $-h(C)$ si cancella con $+h(C)$. Si chiama "somma telescopica". Alla fine il peso del percorso intero cambia solo di una quantità fissa: $+h(A) - h(D)$. Visto che questo "bonus/malus" fisso si applica a *tutti* i percorsi possibili che vanno da A a D , il percorso che era il più veloce prima, rimarrà il più veloce anche dopo! Geniale.

CAPITOLO 16: FLUSSO MASSIMO

La Teoria (Zero Omissioni & Matematica Umana)

Immagina una rete di tubature dell'acqua. C'è una stazione di pompaggio (la Sorgente, s) e un depuratore finale (il Pozzo, t). Ogni tubo ha un diametro massimo, ovvero una **Capacità** $c(u, v)$. Il nostro obiettivo è spingere la quantità massima possibile di acqua (il **Flusso** $f(u, v)$) da s a t .

Perché sia un flusso fisicamente valido, la matematica impone **due regole invalicabili**:

1. **Vincolo di Capacità:** $f(u, v) \leq c(u, v)$. L'acqua non può far esplodere il tubo.
2. **Conservazione del Flusso:** Per ogni incrocio (nodo) che non sia s o t , la somma dell'acqua che entra DEVE essere uguale alla somma dell'acqua che esce ($\sum f(v, u) = \sum f(u, v)$). Niente si crea e niente si distrugge.

1. Metodo di Ford-Fulkerson e le Reti Residue

- *L'Idea:* Parto con i tubi vuoti (flusso 0). Cerco un percorso da s a t dove c'è ancora spazio nei tubi (Cammino Aumentante). Spingo acqua. Ripeto finché tutti i percorsi sono bloccati.
- *Il Motore Magico (La Rete Residua G_f):* Questa è la vera genialità. Quando spingo 10 litri d'acqua da A a B , creo un tubo "fantasma" (residuo) che va all'indietro, da B ad A , con capacità 10.
 - *Matematica Umana:* $c_f(v, u) = f(u, v)$. Perché? Perché questo permette all'algoritmo di "cambiare idea". Se in futuro trova un percorso migliore, può "spingere acqua all'indietro" in questo tubo fantasma, che fisicamente equivale a *cancellare* l'acqua che aveva mandato prima, reindirizzandola altrove.
- *Il Collo di Bottiglia:* In un cammino aumentante, la quantità massima di acqua che posso spingere è dettata dal tubo più stretto di quel percorso ($c_f(p)$).

2. Teorema Max-Flow Min-Cut (Il Teorema Supremo)

- Un "Taglio" è letteralmente una linea immaginaria che taglia il grafo in due metà: una contiene s , l'altra contiene t . La "Capacità del Taglio" è la somma delle capacità di tutti i tubi che vanno dalla metà di s alla metà di t .

- *Il Teorema*: Il Flusso Massimo che puoi spingere in *tutta* la rete è **esattamente uguale** alla capacità del Taglio Minimo (il "collo di bottiglia globale" della rete). Se dimostri che il flusso attuale è uguale a un taglio, hai la certezza matematica di aver finito (il flusso è massimo).

3. Le due varianti dell'Algoritmo

- **Algoritmo di Base (Ford-Fulkerson)**: Cerca i percorsi a caso.
 - *Costo e Caso Pessimo*: Se le capacità sono numeri interi grandi, e l'algoritmo ha sfortuna, aumenta il flusso di 1 litro alla volta. Il tempo esplode a $O(E \cdot |f^*|)$ (dove f^* è il valore del flusso massimo). È l'unico algoritmo il cui tempo dipende dai *valori* scritti dentro i nodi, non solo da quanti sono!
- **Algoritmo di Edmonds-Karp**: Mette una pezza al problema. Trova i cammini usando la **BFS (Visita in Ampiezza)**, prendendo sempre il percorso col minor numero di tubi.
 - *La Magia*: La distanza dei nodi aumenta in modo monotono. Il tempo crolla a una garanzia pura: $O(VE^2)$.

4. Abbinamento Massimo nei Grafi Bipartiti

- *L'Idea*: Hai un gruppo L (es. Operai) e un gruppo R (es. Lavori). Ogni operaio sa fare solo alcuni lavori. Come ne assegni il massimo numero possibile?
- *La Traduzione*: Aggiungi una "Super-Sorgente" s collegata a tutti gli operai, e un "Super-Pozzo" t collegato a tutti i lavori. Tutti gli archi hanno **capacità 1**.
- *La Risoluzione*: Fai girare Edmonds-Karp. Il Flusso Massimo trovato coinciderà perfettamente con l'Abbinamento Massimo! Costo: $O(VE)$.

Lente per le Crocette (La Caccia al Trabocchetto)

Qui le trappole logiche si sprecano, le vecchie prove ne sono piene.

- **Il Trabocchetto dell'Aumentante**: "*In una rete di flusso un flusso è massimo se e solo se non esistono cammini aumentanti*". **VERA**. È letteralmente il punto 2 del Teorema Max-Flow Min-Cut.
- **Il Trabocchetto dei Tagli Non Saturi**: "*Se in una rete di flusso troviamo più di un taglio non saturo allora esiste un cammino aumentante*". **FALSA** (Ed è apparsa nel tuo secondo parziale!). Fai tantissima attenzione alla logica: se il "taglio minimo" (il vero collo di bottiglia) è saturo, il flusso è già bloccato e massimo. Il fatto che *altri* tagli più larghi nella rete non siano ancora saturi non significa nulla! L'acqua non può comunque raggiungerli.
- **L'Esplosione della Rete Residua**: "*La rete residua G_f contiene esattamente lo stesso numero di archi della rete originale G* ". **FALSO**. Può contenerne fino al **doppio** ($2|E|$), perché per ogni tubo reale in cui passa acqua, viene generato un tubo fantasma all'indietro.

- **La trappola dei Grafi Bipartiti:** *"È possibile trovare un abbinamento massimo in un grafo bipartito nel tempo $O(VE)$ ".* **VERA.** Sfrutta esattamente la modellazione a rete di flusso unitaria.

Lente per i Problemi (Il Kit di Costruzione)

Questo capitolo ti sblocca l'ultimo "Maxi-Problema" classico. Ne hai uno identico nel file degli esami passati (Appello del 23 giugno 2023).

- **Lo Scenario:** *"Assegnare k risorse umane a n progetti nei prossimi m mesi"* con vincoli su chi non può lavorare in certi mesi.
- **La Progettazione (Costruire il mattoncino):**
 1. *Traduzione:* "Traduco il problema di assegnazione in un problema di Flusso Massimo su Grafo Bipartito".
 2. *Modellazione Nodi:* Crea un nodo per ogni "Mese" (gruppo di sinistra) e un nodo per ogni "Progetto" (gruppo di destra).
 3. *Super-Nodi:* Aggiungi una Super-Sorgente s collegata a tutti i Mesi. Aggiungi un Super-Pozzo t a cui si collegano tutti i Progetti.
 4. *Le Capacità (Il vero trucco):*
 - Gli archi $s \rightarrow Mese_i$ hanno capacità k (risorse massime disponibili in quel mese).
 - Gli archi $Mese_i \rightarrow Progetto_j$ hanno capacità 1 (una risorsa lavora a un progetto al mese), ma inserisci l'arco *solo se* non ci sono vincoli contrari.
 - Gli archi $Progetto_j \rightarrow t$ hanno capacità pari ai "mesi risorsa" necessari per finire quel progetto.
 5. *La Soluzione e i Costi:* "Lancio l'algoritmo di Edmonds-Karp. Se il flusso massimo $|f|$ equivale alla somma totale dei mesi risorsa richiesti da tutti i progetti, la tabella delle allocazioni è possibile e si deduce guardando gli archi con flusso pieno. La complessità è $O(VE^2)$ "* (oppure $O(VE)$ se lo modelli come abbinamento puramente unitario dividendolo ulteriormente).

Prevenzione delle Sorprese (Oltre i PDF)

Il tuo professore potrebbe spingersi oltre con due variazioni tattiche letali su cui molti studenti cadono in fase di orale o nell'ultimo problema dello scritto:

- **Il Problema a Sorgenti e Pozzi Multipli:** *"Cosa succede se abbiamo 5 fabbriche (sorgenti) e 3 magazzini (pozzi)?"*. La risposta infallibile: non si modella l'algoritmo su sorgenti multiple (diventerebbe un incubo). **Si crea una singola "Super-Sorgente" fantasma** collegata alle 5 fabbriche con archi di capacità ∞ , e un

"Super-Pozzo" fantasma collegato ai 3 magazzini con capacità ∞ . Poi si lancia il normale Edmonds-Karp!

- **Archi Antiparalleli:** Se nel grafo originale ci sono tubi che vanno in entrambe le direzioni tra due stessi nodi ($A \rightarrow B$ e $B \rightarrow A$), la matematica delle reti residue (che crea archi fantasma opposti) va in cortocircuito. Il rimedio? Spezzare uno dei due archi originari mettendoci un nodo finto in mezzo ($A \rightarrow X \rightarrow B$). In questo modo, l'algoritmo standard funziona senza accavallamenti matematici.